코드잇 스프린트 AI 15기 · 초급 프로젝트 · 3팀



알약몬

찍으면 기록되고, 기록하면 캐릭터가 자라는 복약 서비스

실험 결과 보고서

최종 모델	Kaggle mAP@[0.75:0.95]	베이스라인 대비	배포
YOLO11m	0.63060	+111%	Flutter 웹 앱

3팀 윤동욱(팀장) · 권영현 · 강재민 · 장영훈

프로젝트 기간 2026.10.20 - 2026.11.05 | 작성 2026.11.05 · 수정 2026.11.11

모델 저장소 · 앱 저장소 · Kaggle 대회

초록

알약몬은 알약 사진 한 장으로 복용을 기록하고, 그 기록을 캐릭터 성장과 질병 몬스터 퇴치라는 보상으로 이어 주는 1020 대상 복약 서비스다. 이 보고서는 그 핵심인 검출 모델을 다룬다. 우리는 사진 속 알약마다 위치와 종류를 찾는 검출 모델을 만들고, 기기 안에서 추론하는 Flutter 웹 앱으로 배포했다. 대회 채점 대상은 56종이고, 앱 모델은 AI Hub 외부 전용 알약 40종을 더해 96종을 찾는다. 가장 큰 제약은 학습 사진 232장과 심한 클래스 불균형이었다. 그래서 AI Hub 외부 데이터로 희귀 클래스를 클래스당 157 bbox까지 채웠고, test에 대회 밖 알약이 있다는 것을 확인한 뒤에는 그 알약을 새 클래스로 학습했다. 그 결과 YOLO11m의 Kaggle mAP@[0.75:0.95]는 **0.29946에서 0.63060으로 2.1배**가 됐다. 같은 과정으로 YOLO26m은 0.60721, Faster R-CNN은 0.45759를 얻었다. 베이스라인에서는 YOLO26m이 0.18로 가장 낮았지만, 외부 데이터를 같은 방식으로 넣자 YOLO11m과 같은 수준(0.45379 대 0.44885)이 됐다. 외부 데이터를 넣은 뒤의 증강·라벨 재검수·각인 OCR은 Kaggle 점수를 -0.027에서 +0.014 사이로 움직였다. 데이터 쪽 효과(+0.1 이상)보다 한 자릿수 작았고, seed 하나로 잰 값이라 ±0.01 안의 차이는 노이즈와 구분되지 않는다. 이 데이터에서는 모델이나 증강보다 데이터의 양과 구성이 성능을 결정했다. 최종 모델은 우리가 낸 제출 중 최고점이고 앱 연동을 이미 마친 YOLO11m으로 정했다. 두 YOLO 모델의 정확도 우열은 이 실험들로 확정하지 못했다.

우리가 내린 결정의 흐름



목차

초록 2

목차 3

1장. 서론 및 프로젝트 개요 5

1.1 프로젝트 소개 5

1.2 문제 정의 6

1.3 배포: 모델을 앱으로 6

1.4 팀과 일정 7

2장. 데이터 처리 및 EDA 8

2.1 데이터 구성 8

2.2 데이터 전처리 과정 8

2.3 EDA 분석 내용 9

2.4 클래스 불균형 시각화 11

2.5 데이터 품질 개선 과정 12

3장. 모델 선정 및 비교 14

3.1 모델 선정 방향 14

3.2 핵심 근거 논문 1 14

3.3 핵심 근거 논문 2 15

3.4 모델 아키텍처 16

3.5 최종 선정: YOLO11m 17

4장. 실험 과정 및 결과 19

4.1 공통 설정 19

4.2 베이스라인 및 아키텍처 비교 19

4.3 데이터 품질 개선 실험 21

4.4 데이터 증강 기법 비교 22

4.5 각인 OCR 결합 실험 23

4.6 모델별 성능 비교 24

4.7 최종 모델 성능 25

5장. 결과 분석 및 시각화 27

5.1 무엇이 점수를 올렸나 28

5.2 검증 점수를 믿을 수 없었던 이유 29

5.3 최종 설정 위 추가 실험 30

5.4 혼동행렬과 과적합 30

5.5 서비스에 넣어도 되는가 32

5.6 종합 33

6장. 결론 및 향후 과제 34

6.1 결론 34

6.2 우리가 배운 것 34

6.3 한계 34

6.4 향후 과제 35

부록 36

A. 사용 기술 36

B. 근거 PR 36

C. 참고 자료 36

D. 수정 이력 36

1장. 서론 및 프로젝트 개요

한 일	사진으로 복용을 기록하고 보상으로 기록을 이어 가게 하는 서비스를 정하고, 그 핵심인 알약 검출 모델을 Flutter 웹 앱으로 배포했다.
이유	복약 기록은 귀찮아서 끊기기 쉽고, 기록만 하는 앱은 이미 많기에 게임 보상형으로 차별화를 설계했다.
결과	정확도(mAP@[0.75:0.95])를 주 지표로, 속도를 보조 기준으로 정했고, 모델은 기기 안에서 돈다.

1.1 프로젝트 소개

알약몬은 알약 사진을 찍으면 어떤 약인지 알아보고, 찍은 시각에 그 약을 먹은 것으로 기록한다. 처음에는 헬스케어 기록 앱으로 출발했지만, 기록만 해 주는 앱은 이미 많고 기록은 금방 끊긴다. 그래서 우리는 꾸준히 기록을 이어가도록 **게임형 보상을 마련하여** 기존 앱과의 한계를 개선하고 차별성을 설계했다. 주 사용자를 게임형 보상에 익숙한 **10-20대**로 잡았다. 영양제도 대상에 넣어 환자가 아닌 사람도 매일 쓰게 했다.

알약몬 보상 구성(설계안)

구성	내용
복용 일정 등록	처방전을 스캔하면 OCR로 약 이름·복용 기간·하루 횟수·1회 알 수를 읽고, 사용자가 확인하면 일정이 된다
비타민 · 영양제	정해진 일정대로 기록하면 내 캐릭터가 자라고, 연속 기록 업적으로 스킨·액세서리 뽑기를 얻는다
치료약	처방전을 등록하면 질병 몬스터가 나오고, 하루치를 기록할 때마다 체력이 준다. 복용 기간을 마치면 퇴치 보상과 몬스터 펫을 받는다
하루 기준량 제한	처방 하루 복용량(영양제는 권장량)을 넘은 기록은 보상에 반영하지 않는다. 보상이 과복용을 부추기지 않게 한다

- **정확도가 우선**: 어떤 약을 몇 알 먹었는지가 기록, 하루 기준량 판정, 보상을 모두 결정한다. 그래서 박스가 꼭 맞아야 점수가 나는 mAP@[0.75:0.95]를 주 지표로 정했다.
- **속도도 중요**: 휴대폰에서 바로 결과가 나와야 한다. 그래서 정확도가 비슷하면 one-stage 모델을 고른다.
- **약통이 아니라 알약을 본다**: 약통 라벨은 믿을 수 없어 알약 자체를 인식한다.

1.2 문제 정의

문제 정의

항목	내용
서비스 문제	복약 기록은 귀찮아서 끊긴다. 기록은 사진으로 자동화하고, 이어 갈 이유는 보상으로 만든다
입력 / 출력	알약 사진 1장(알약 2-4개) / 알약마다 bbox, 약 코드, 확신도
평가 지표	mAP@[0.75:0.95]: IoU 0.75-0.95 다섯 기준 AP의 평균
대회	Kaggle ai15-level1-project, test 842장
제약	train 232장 · bbox 763개 · 56종. 클래스 불균형이 크고 같은 장면을 각도만 바꿔 찍은 사진이 많다

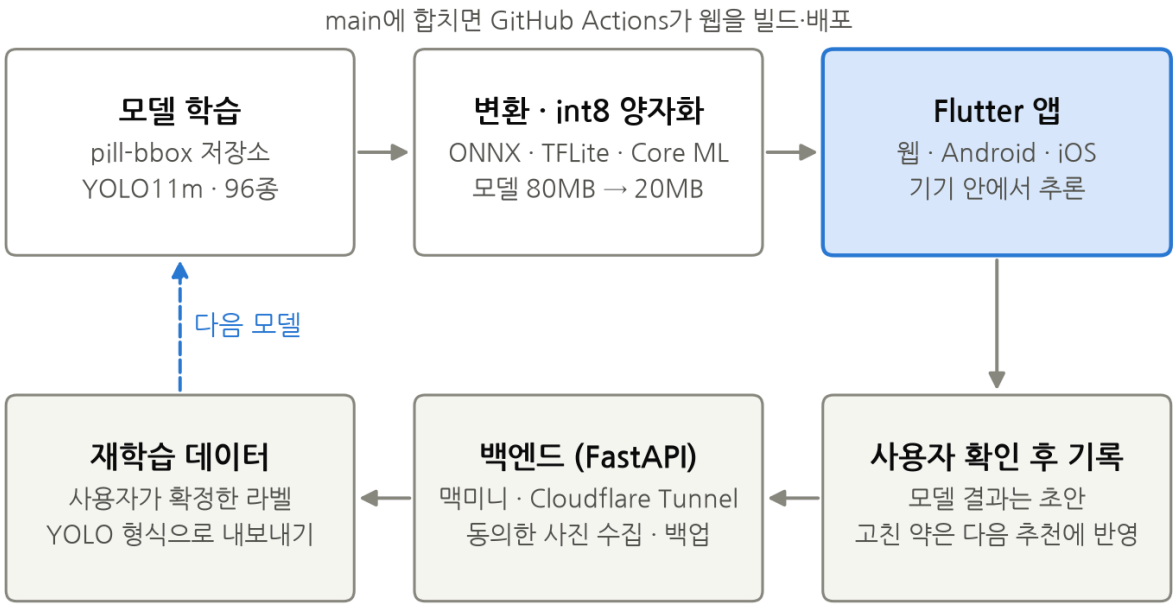
대회 페이지: Kaggle ai15-level1-project

1.3 배포: 모델을 앱으로

우리는 학습한 모델을 **기기 안에서 추론하는 Flutter 웹 앱**으로 배포했다(Android·iOS는 아직 검증하지 않았다, 앱 저장소). 기기 안에서 돌리기로 한 것은 서버가 꺼져도 기록할 수 있고, 사진이 기기 밖으로 나가지 않기 때문이다.

모델 배포와 개선 순환

목적: 학습한 모델이 사용자에게 닿고, 다시 학습 데이터로 돌아오는 경로를 보인다. 읽는 법: 위 줄이 배포, 아래 줄이 사용자 데이터 수집이다.



배포 구성

항목	내용
모델	YOLO11m(대회 56종 + AI Hub 40종 = 96종). int8 양자화로 80MB → 20MB, 탐지 결과는 원래 모델과 같음(신뢰도 차이 최대 0.04)
추론	웹 onnxruntime-web(Web Worker), Android TFLite, iOS Core ML
배포	웹 먼저. main에 합치면 GitHub Actions가 빌드해 맥미니에 올리고 Cloudflare Tunnel로 공개
오인식 대비	신뢰도 0.5 미만은 버림. 한 알약에 두 약이 0.7 넘게 겹치면 하나만 남김. 놓친 물체는 색으로 찾아 "알 수 없는 물체"로 표시(대회 사진 알약의 99% 검출)
백엔드	FastAPI + SQLite. 동의한 사용자의 사진과 확정 라벨을 모으고 기록을 백업

1.4 팀과 일정

팀 구성

이름	맡은 일
윤동욱 (팀장)	PM
권영현	EDA, 모델링 (YOLO11m)
강재민	EDA, 모델링 (YOLO11m)
장영훈	모델링 (YOLO26m, Faster R-CNN)

2장. 데이터 처리 및 EDA

한 일	라벨 오류를 고치고, 같은 장면이 학습·검증에 나뉘지 않게 조합 단위로 나누고, AI Hub 데이터로 희귀 클래스를 채웠다.
이유	232장 중 같은 장면이 많아 무작위 분할은 점수를 부풀리고, 희귀 약 18종은 bbox가 5개 이하였다.
결과	bbox 763 → 771개(v1), 누수율 90.9% → 조합 단위, 클래스 불균형 CV 1.534 → 0.100.

2.1 데이터 구성

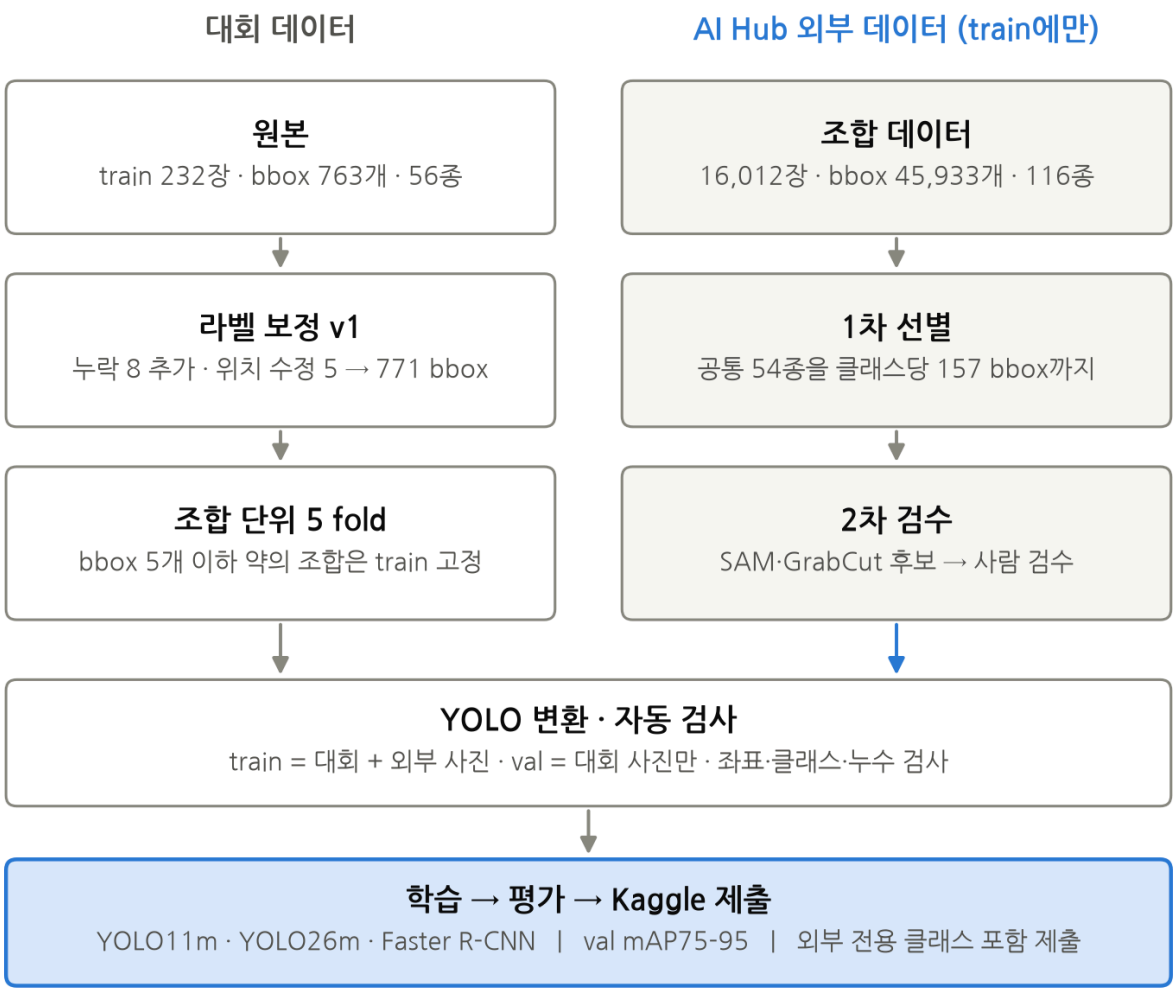
데이터 구성

구분	내용
대회 train	232장, bbox 763개, 56종. 976×1280, 연회색 배경, 각도 70°·75°·90°
대회 test	842장 (Kaggle 채점)
AI Hub	경구약제 조합 데이터 16,012장, bbox 45,933개, 116종. 대회와 공통 54종, 외부 전용 62종

외부 데이터: AI Hub 경구약제 이미지 데이터

2.2 데이터 전처리 과정

모든 실험이 같은 라벨·같은 분할을 쓰도록 데이터를 파이프라인 하나로만 만들었다. 원본 JSON은 고치지 않고 보정 기록(corrections.csv)으로 적용하며, 단계마다 자동 검사가 어긋나면 멈춘다.

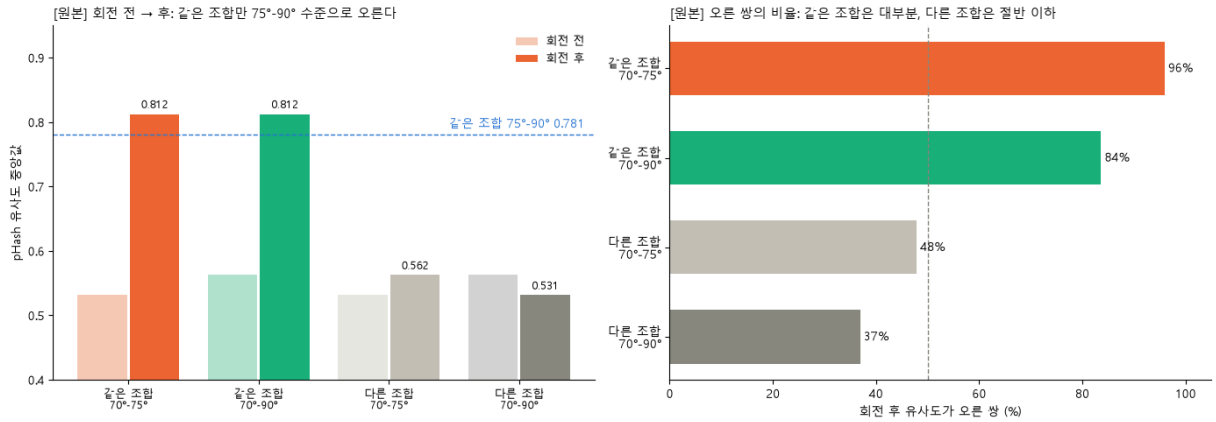


2.3 EDA 분석 내용

- **같은 장면의 반복**: 232장은 약 조합 114개를 1-3장씩 찍은 것이다. 특히 70° 사진은 180° 뒤집혀 있어, 돌리면 75도와 90도랑 pHash 유사도가 0.531에서 0.812로 오른다. 즉 사실상 하나의 데이터에 가깝다고 말할 수 있다.



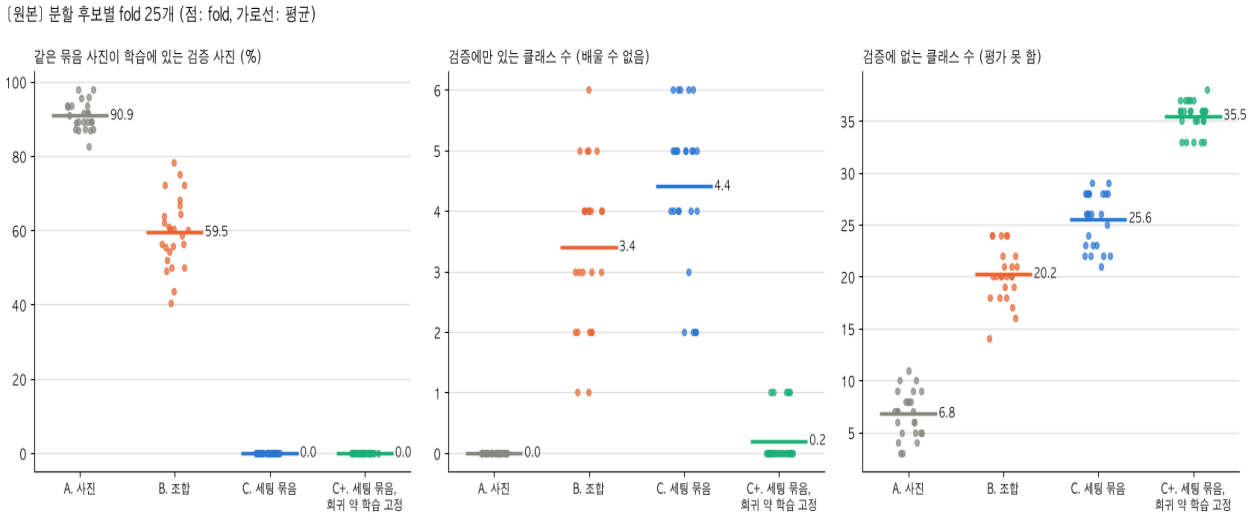
- pHash를 보다가 이상한 점을 찾았다. **조합이 다른데도** 유사도가 0.9 이상인 쌍이 62개, 1.000인 쌍도 1개 있었다. 사진을 열어 보니 중복 사진이 아니었다. 같은 촬영 세팅에서 알약 일부만 바꿔 찍은 사진이었다. 즉 조합으로 데이터셋을 분할하면 검증 누수가 발생할 수 있다.



- **세팅 공유**: 조합이 달라도 공통 약 2개 이상이 같은 자리에 놓인 사진이 103쌍이다. 무작위로 나누면 같은 장면이 학습과 검증에 함께 들어간다. 그래서 "공통 약 2개 이상이 같은 자리(30px 이내)에 있으면" ***세팅 공유***라는 기준을 세우고 박스 위치로 다시 셌다.

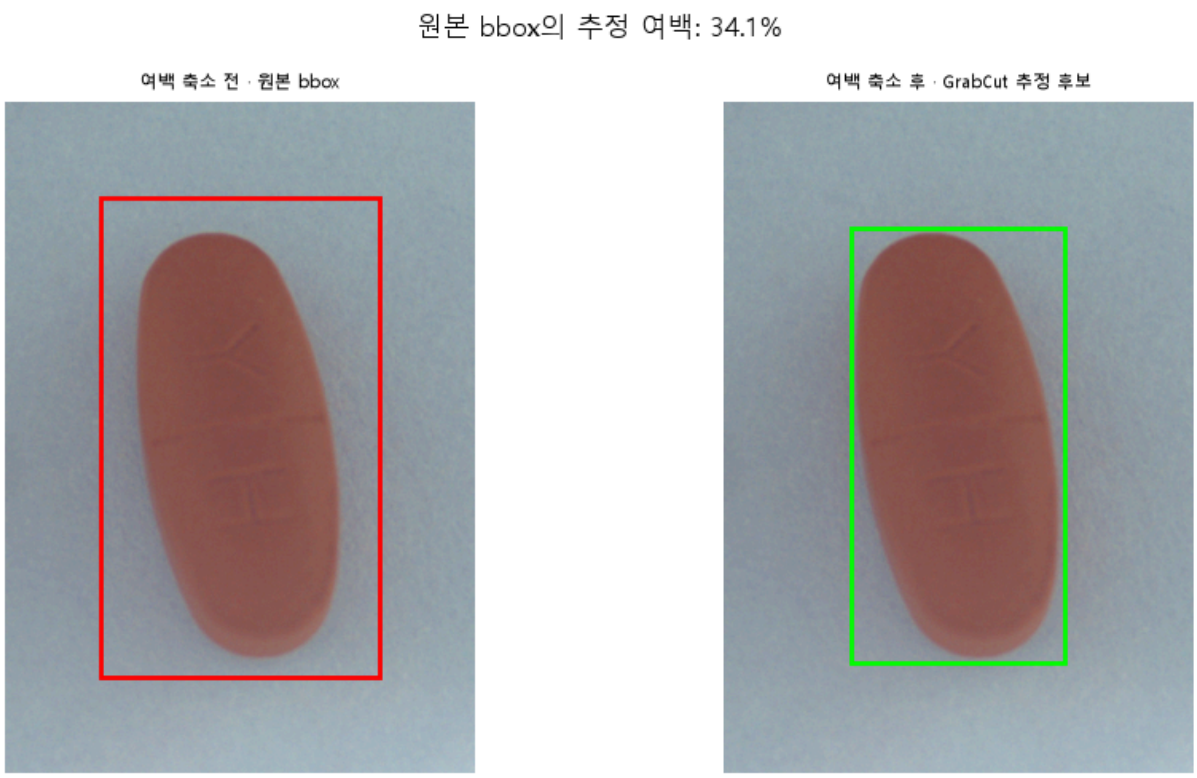
> **→ 같은 YOLO11m을 두 방식으로 학습해 보니, 세팅 단위 분할의 검증 점수(0.6047)가 조합 단위(0.8061)보다 0.20 낮았다. 세팅 공유 누수가 실제로 점수를 부풀린다는 근거다. 하지만 세팅 단위에서는 베이스라인 점수가 너무 낮게 나왔고 희귀 약을 배울 수 없거나 평가할 수 없게 된다.**

조합 vs 셋팅 별 누수와 클래스 누락



그래서 최종 분할은 조합 단위로 나누되 희귀 약(bbox 5개 이하)이 든 조합은 train에 고정했다.

- **bbox 여백**: 정답 박스는 알약보다 사방 약 8px 넓다. 747개가 같은 경향이라 일관된 기준으로 보고 유지했다.



2.4 클래스 불균형 시각화

3351 한 종이 bbox의 20.4%(157개)인 반면, bbox 5개 이하인 약이 18종(32%)이다. 최대/최소가 52배(157개 대 3개)이고, 희귀 약 18종 중 17종은 한 조합에서만 나온다. 우리는 불균형을 증강이 아닌 실제 사진으로 먼저 줄이기로 하고, AI Hub에서 가져오는 방법 다섯 가지를 비교했다.

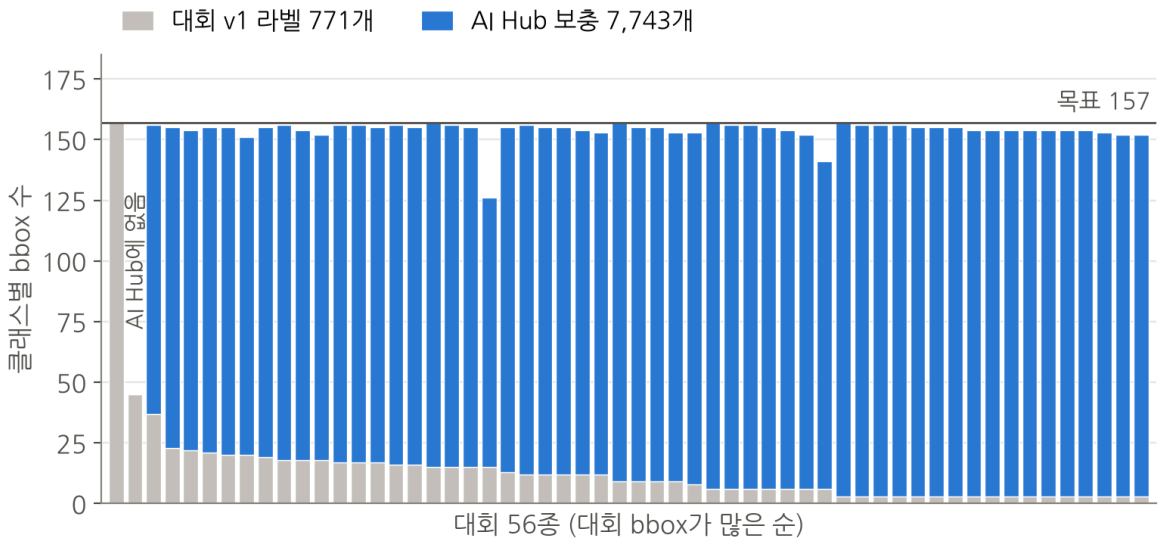
외부 데이터 보충 방법 비교 (CV: 낮을수록 균등)

방법	추가 대회 bbox	동반 외부 전용 bbox	CV (56종)
추가 전	0	-	1.534
공통 54종을 클래스당 157 bbox까지 (채택)	7,803	새 클래스로 학습	**0.100**
대회 클래스만 있는 사진	848	0	1.136
외부 전용 1개 이하 + CV 최소화	4,093	1,320	0.310
혼합 원본 사진 전체	13,257	7,267	0.518

CV(변동계수)는 클래스별 bbox 수의 표준편차/평균이다. 157은 외부에 없는 최다 클래스 3351의 개수다.

> → **그래서** CV가 가장 낮고 test에 있을 외부 알약도 함께 들어오는 첫번째(157bbox) 안을 골랐다.

AI Hub 보충으로 희귀 클래스를 157개 가까이 채웠다

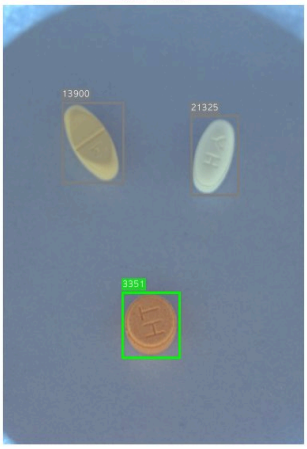


외부 데이터(ai_hub) 선별과 검수. 공통 54종을 클래스당 157개까지 seed 42로 뽑고, 좌표 오류·bbox 겹침·크기 불일치·대회와 같은 조합(검증 누수 가능)인 사진을 뺐다. 최종 5,842장(외부 bbox 21,720개, 외부 전용 40종)을 train에 더했다. 2차로 박스가 알약을 다 감싸지 못한 3,109개를 골라 SAM·GrabCut으로 후보를 만들고, 사람이 검수해 bbox 223개를 고치고 106장을 뺐다.

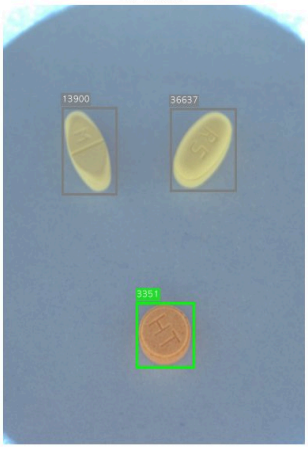
2.5 데이터 품질 개선 과정

대회 라벨 보정(v1). 누락된 라벨 8개를 추가해 bbox를 763개에서 771개로 만들었다. 위치가 틀린 박스 5개(다른 약과 좌표가 겹친 3개, 이미지 밖 좌표 1개, 빈 배경 박스 1개)와 잘린 캡슐 박스 1개는 위치를 고쳤다.

〔보정〕 K-003351-013900-021325 (70°)
라벨 누락: 3351



〔보정〕 K-003351-013900-036637 (70°)
라벨 누락: 3351



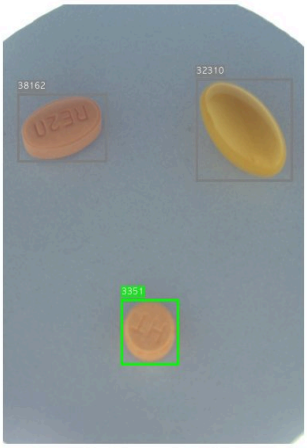
〔보정〕 K-003351-020014-022074 (90°)
라벨 누락: 20014



〔보정〕 K-003351-021325-032310 (90°)
라벨 누락: 32310



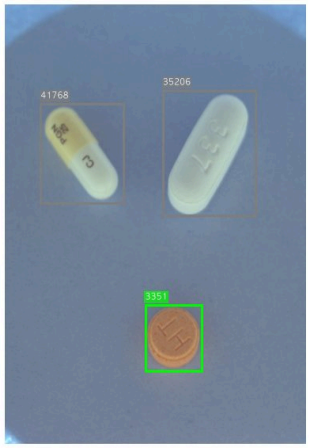
〔보정〕 K-003351-032310-038162 (70°)
라벨 누락: 3351



〔보정〕 K-003351-033880-038162 (75°)
라벨 누락: 33880



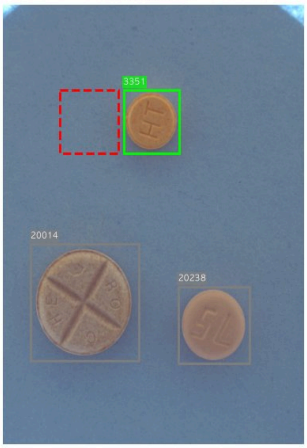
〔보정〕 K-003351-035206-041768 (70°)
라벨 누락: 3351



〔보정〕 K-003544-004543-012247-016548 (90°)
라벨 누락: 4543



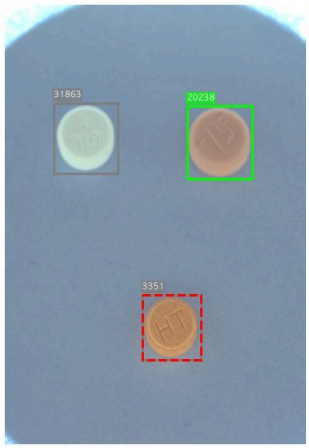
〔보정〕 K-003351-020014-020238 (90°)
빈 배경 박스: 3351



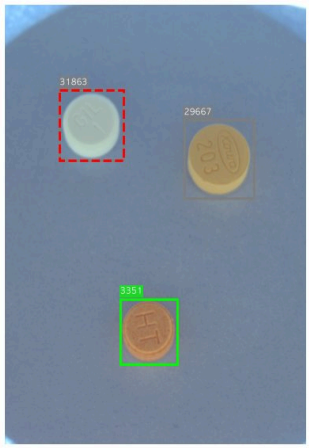
〔보정〕 K-001900-016548-019607-033009 (70°)
다른 약과 좌표 중복: 16548



〔보정〕 K-003351-020238-031863 (70°)
다른 약과 좌표 중복: 20238



〔보정〕 K-003351-029667-031863 (70°)
다른 약과 좌표 중복: 3351



3장. 모델 선정 및 비교

한 일	one-stage 두 개(YOLO11m, YOLO26m)와 two-stage 하나(Faster R-CNN)를 같은 데이터 전략과 지표로 비교했다. 다만 분할과 실행 환경은 모델마다 달랐다(3.5). **선정 기준과 후보는 알약 인식·객체 검출 논문을 먼저 읽고 정했다.**
이유	약은 정확도가 우선이지만 휴대폰에서 바로 돌아야 해서, 정확도와 속도·변환 편의를 함께 봐야 했다.
결과	우리가 낸 제출 중 최고점(0.63060)이고 앱 연동을 이미 마친 YOLO11m을 골랐다. 두 YOLO의 정확도 차이(0.023)는 조건이 달라 확정하지 못했다.

3.1 모델 선정 방향

우리는 모델을 고르기 전에 논문부터 읽었다. 알약 인식 연구 1편과 검출기 비교·설계 논문들에서 "어떤 기준으로 무엇을 비교해야 하는지"를 먼저 정했다. 아래 네 기준은 모두 논문 근거가 있다.

- **높은 IoU 정확도**: 박스가 IoU 0.75 이상 맞아야 점수가 난다. COCO 벤치마크는 IoU 0.5-0.95의 여러 기준에서 AP를 평균하는 mAP@[.5:.95]로, 박스를 정밀하게 맞추는 모델이 높은 점수를 받도록 했다.
- **휴대폰 속도와 변환**: 기기 안 추론을 위해 가볍고, TFLite·Core ML·ONNX로 바꾸기 쉬워야 한다. 검출기의 속도·정확도 트레이드오프를 같은 조건에서 비교한 연구에서는, two-stage(Faster R-CNN)가 더 정확한 대신 느렸고 one-stage가 빨랐다.
- **적은 데이터**: COCO 사전학습 가중치로 전체 파인튜닝한다.
- **one-stage와 two-stage 비교**: 약은 정확도가 우선이라 two-stage도 같은 조건으로 비교했다. one-stage는 배경 후보가 압도적으로 많은 클래스 불균형 때문에 two-stage보다 정확도가 낮다고 알려졌다. 하지만 Focal Loss 이후의 one-stage는 그 차이를 거의 따라잡았다. 이 부분은 해당 논문만으로 단정할 수 없어서 직접 비교하기로 했다.

3.2 핵심 근거 논문 1: Ashraf 등, "Code-Based Versus AutoML Methods for Pill Recognition in Clinical Settings" (JMIR Medical Informatics, 2026)

이 연구는 헝가리 병원 약제부에서 수작업 조제에 쓰는 약통(dosette box)에 알약을 넣고 찍은 사진으로, 자주 조제하는 약 30종을 인식하는 모델을 만들었다. 코드 기반 YOLO11과 클라우드 AutoML 3종(Amazon, Google, Microsoft)을 비교했다. 학습 사진 수가 성능에 주는 영향을 보려고, 로그 간격으로 사진 수를 늘린 하위집합 5개로 각각 학습했다. 테스트셋 하나는 한 사진에 알약이 여러 개 있어 우리 과제와 비슷하다

논문에서 본 것	우리가 한 결정
속도·정확도·학습 시간의 균형 때문에 중간 크기인 **YOLO11-m**을 썼다	후보를 m 크기로 맞췄다. l·x는 프로젝트 규모상 휴대폰에 넣기에 크다고 봤다

논문에서 본 것	우리가 한 결정
학습 사진을 1,230장에서 26,880장까지 늘렸을 때, **YOLO11이 데이터에 비례해 가장 꾸준히 올랐다**. 가장 어려운 임상 데이터셋에서 정확도는 22.71%에서 64.58%로 올랐다	우리 데이터(232장)가 병목이라고 보고, **AI Hub 외부 데이터로 학습량을 늘리는 전략**(2.4, 4.3)의 근거로 삼았다.
정확도뿐 아니라 **놓친 알약 비율(FNR)을 임상 안전 지표**로 따로 봐야 한다고 했다. 이 논문은 mAP@0.5와 mAP@0.5-0.95를 썼다.	캐글 대회 지표인 더 엄격한 mAP@[0.75:0.95]를 주 지표로 쓰고, 혼동행렬에서 **놓친 알약 수를 따로 확인**했다.

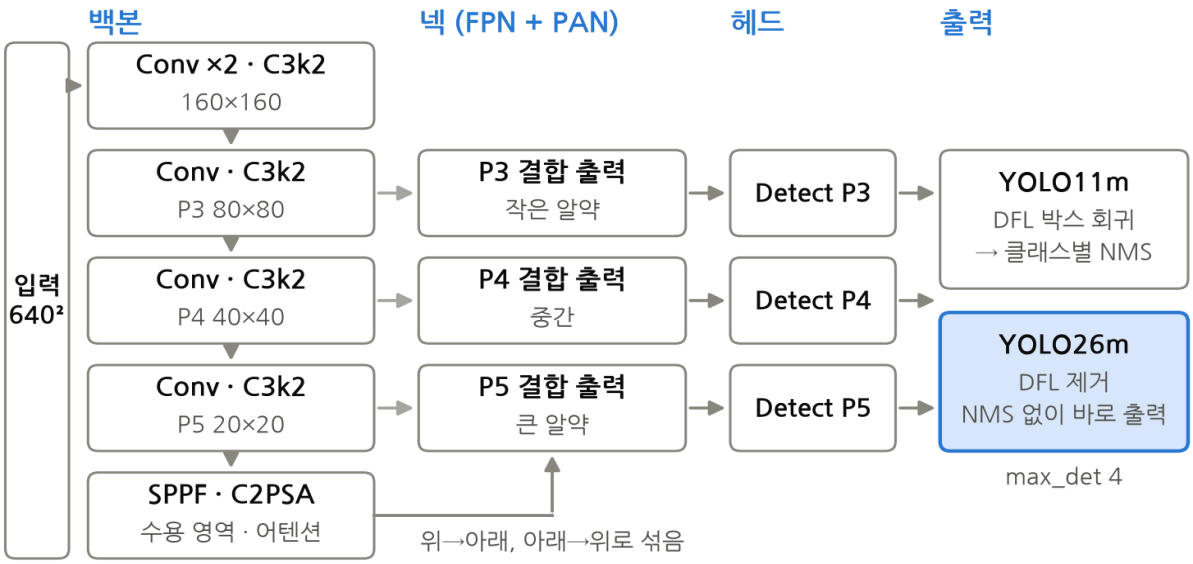
해당 논문에선 YOLO11은 모든 데이터셋에서 1위는 아니었지만 항상 최상위권이었다. 학습 데이터가 늘수록 선형적으로 오른다는 것이 핵심적인 장점으로, 가장 큰 학습셋에서 정확도 80.83%, 정밀도 1.0, 재현율 0.96에 도달했다. 반면 최적 성능에는 더 큰 학습 데이터가 필요할 수 있다고 했다.

3.3 핵심 근거 논문 2: Yurdakul & Çakmak, "Multi-Scale Performance Benchmarking of YOLO Models for Effervescent Tablet Defect Detection" (Preprints.org, 2026)

첫 번째 논문으로 "YOLO11, 그리고 m 크기"라는 방향을 정한 뒤, 우리는 **YOLO11과 YOLO12, 그리고 최신 YOLO26 중 무엇을 쓸지** 정해야 했다. 두번째 논문은 정제(알약) 사진에서 YOLO11·YOLO12·YOLO26을 크기별(n·s·m·l·x)로 15개 모두 같은 조건에서 비교한 연구다. 대신 본 논문은 동료 심사 전이라는 한계점을 가지고 있다.

논문에서 본 것	우리가 한 결정
YOLO11이 정확도와 속도를 모두 이끌었다. 최고 정확도는 YOLO11l(mAP@0.5 96.8%, mAP@0.5:0.95 91.7%)이었다. 가장 효율적인 것은 YOLO11n(mAP@0.5 95.6%, 파라미터 250만 개, 345.9 FPS)이었다	첫번째 논문과 같은 결과가 일치했기에, YOLO11을 기준 모델로 삼았다
크다고 항상 좋지 않았다. YOLO11의 mAP@0.5는 n 95.6%에서 l 96.8%까지 오른 뒤 x에서 96.6%로 오히려 내렸다	L까지 키우지 않고, 첫 번째 논문처럼 속도와 정확도의 중간인 m 크기로 정했다
YOLO12는 정확도는 경쟁력 있지만, 크기가 커질수록 추론이 눈에 띄게 느렸다. attention 블록 A2C2f를 많이 쓰기 때문이다	휴대폰 속도가 중요해 YOLO12는 후보에서 뺐다
모양이 비슷한 클래스끼리 오류가 많았다	혼동행렬을 반드시 보기로 했다(5.4)
학습 설정: 640×640, 100 에폭, 모든 모델 같은 하이퍼파라미터	모델을 바꿔도 하이퍼파라미터는 고정했다(4.1). 베이스라인은 30 에폭으로 시작해 이후 100 에폭으로 맞췄다. 다만 YOLO26m은 대회 사진만으로 100 에폭을 돌리지 않았다(4.2)
증강: 뒤집기·작은 회전·크기·이동, HSV, Mosaic	같은 종류를 쓰되, 각인과 색이 약을 구분하는 단서라 뒤집기는 끄고 회전·색조는 약하게 두었다(4.1)

3.4 모델 아키텍처



YOLO 계열의 구조도 이 논문으로 이해했다

- **YOLO11**: 백본은 다운샘플링 Conv와 C3k2 블록이 번갈아 쌓이고, 끝에 SPPF와 C2PSA가 있다.
  - C3k2는 꼼꼼히 보는 경로와 그대로 넘기는 지름길 경로를 합쳐 가볍게 특징을 뽑는다.
  - SPPF는 여러 배율로 주변을 한 번에 본다.
  - C2PSA는 사진의 각 부분이 다른 부분과 얼마나 관련 있는지 따진다.

넥은 위→아래 경로와 아래→위 경로로 특징을 섞어 세 크기의 특징 맵을 만든다. 헤드는 작은·중간·큰 알약을 나눠 맡는다. YOLO11m은 YOLOv8m보다 파라미터가 약 22% 적다.

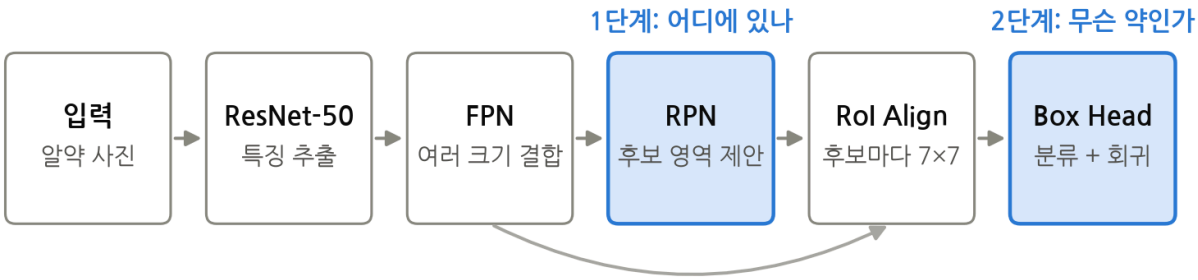
- **YOLO12**: 흐름은 같지만 백본 뒤쪽과 넥의 블록을 대부분 A2C2f(C2f + 영역 주의)로 바꿨다. 관계를 더 잘 보지만 계산이 늘어 큰 크기에서 느려진다.
- **YOLO26**: 백본과 넥은 YOLO11과 거의 같다. C2PSA의 출력이 넥의 위·아래 경로 양쪽으로 이어져, 작은 물체와 큰 물체 모두 전체 맥락을 얻는다. 이 논문이 인용한 YOLO26 분석 논문(Sapkota 등, arXiv:2509.25164)에 따르면 바뀐 점은 셋이다.
  - **DFL 제거**와 **NMS 없는 출력**으로 모델 변환이 쉽고 CPU에서 빨라졌다.
  - 새 옵티마이저 **MuSGD**를 쓴다.
  - 손실 함수가 개선됐다.

우리 공통 학습 설정의 MuSGD(4.1)도 여기서 왔다.

백본이 80·40·20 크기의 특징을 뽑고, 넥이 이를 섞어 작은 알약부터 큰 알약까지 세 크기에서 예측한다. 사진당 알약이 최대 4개라 max_det를 4로 두었다.

Faster R-CNN 구조

목적: two-stage가 YOLO와 다른 점을 보인다. 읽는 법: 파랑 두 칸이 두 단계다.



> *핵심: 후보 영역을 먼저 찾고, 그 영역만 다시 분류·회귀해 박스 정밀도를 높인다.*

3.5 최종 선정: YOLO11m

두 YOLO 후보를 기준별로 다시 놓으면 다음과 같다. 점수는 모두 Kaggle mAP@[0.75:0.95]다.

기준	YOLO11m	YOLO26m
최고 제출	**0.63060**	0.60721 (copy-paste 포함)
같은 데이터 전략 (AI Hub, 외부 알약 = 배경)	0.44885	**0.45379**
분할	combofix 5 fold	combo 단일 holdout (train 193장, val 39장)
변환 구조	DFL과 NMS 후처리가 필요하다	DFL이 없고 NMS 없이 출력해 변환이 더 쉽다(3.4)
앱 연동	웹(ONNX) 연동과 검증을 마쳤다. Android·iOS는 검증하지 않았다	하지 않았다

- 최고 제출의 차이 0.023은 분할·증강·GPU가 서로 다르고 seed 하나로 한 번씩 돌린 결과다. 같은 데이터 전략에서는 오히려 YOLO26m이 0.005 높았다. 그래서 **이 차이를 정확도 우열로 보지 않는다.**
- 변환 구조만 보면 YOLO26m이 유리하다. 따라서 "변환이 쉬워서"는 YOLO11m을 고른 근거가 될 수 없다.
- Faster R-CNN(최고 0.45759)은 차이가 컸다. 다만 증강을 YOLO 기준으로 맞춰 실험해 손해를 봤을 수 있어, 결론이 아니라 향후 과제로 둔다(6.4).

> → **그래서** YOLO11m을 고른 근거는 "더 정확해서"가 아니라 ① 우리가 낸 제출 중 최고점이었고 ② 앱 연동과 검증을 이미 마쳤기 때문이다. 정확도 우열을 확정하려면 같은 분할·seed 3개로 다시 비교해야 한다(YOLO26m 제외 결정 재검토 6절).

4장. 실험 과정 및 결과

한 일	기준에서 한 번에 한 가지만 바꾸는 방식으로 학습량, 외부 데이터, 증강, OCR을 차례로 시험했다.
이유	무엇이 점수를 움직이는지 분리해서 알아야 다음 실험을 고를 수 있었다.
결과	학습량(+0.115), 외부 데이터(+0.034), 외부 전용 알약(+0.182)만 크게 올렸다. 나머지는 -0.027에서 +0.014 사이였다. seed 하나로 잰 결과라 ±0.01 안의 차이는 노이즈와 구분되지 않는다.

4.1 공통 설정

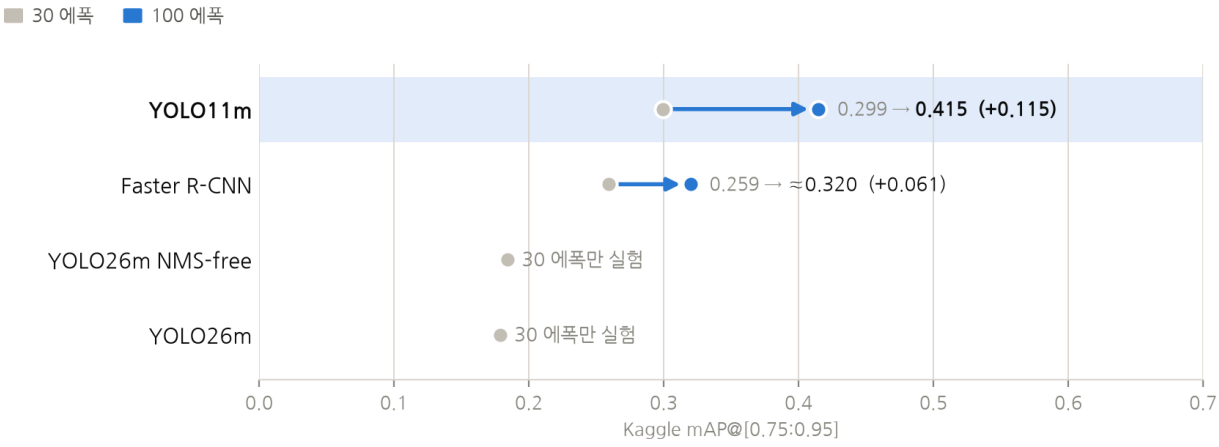
공통 실험 설정

항목	값
학습	640px, 배치 16(메모리에 따라 8), 최대 100 에폭, patience 10, seed 42
옵티마이저	MuSGD, lr0 0.001 선형 감소, warmup 3 에폭 (auto는 항상 AdamW를 골라 고정)
증강 on	회전 10°, 뒤집기 끔, 색조 0.005, Mosaic 1.0 (각인과 색을 지키도록 약하게)
평가	conf 0.001, NMS IoU 0.5, max_det 4. best.pt와 조기 종료는 mAP75-95 기준
제출	A안: 외부 전용 예측 버림 / B안: 외부 전용 예측을 약 코드로 제출

4.2 베이스라인 및 아키텍처 비교

베이스라인 (대회 데이터만)

30 에폭 → 100 에폭으로 늘렸을 때 Kaggle 점수. 학습량만 늘려도 YOLO11m이 +0.115 올랐다.



30 에폭 베이스라인에서는 YOLO11m이 가장 높았다. 30 에폭은 학습이 덜 됐다고 보고 100 에폭으로 늘리자 YOLO11m이 Kaggle +0.115로 크게 올랐다.

YOLO26m은 0.18로 가장 낮았고, 평가 헤드를 NMS에서 NMS-free로 바꿔도 0.17877 → 0.18437로 그대로였다. 우리는 이 결과만으로 10.31에 YOLO26m을 후보에서 뺐다가, 근거가 부족하다는 피드백을 받고 11.02에 다시 넣었다. 두 모델이 거친 단계를 나란히 놓으면 베이스라인의 차이는 모델 구조보다 학습량 부족으로 설명된다.

단계 (Kaggle mAP@[0.75:0.95])	YOLO11m	YOLO26m	근거
① 대회 사진만, 30 에폭	0.29946	0.17877	#101, #102
② 대회 사진만, 100 에폭	0.41485	돌리지 않음	#103, #105
③ + AI Hub, 외부 알약 = 배경	0.44885	**0.45379**	#105, #112
④ 외부 알약을 새 클래스로 학습·제출	**0.63060**	0.60721	#109, #114

- **평가 헤드는 원인이 아니었다.** 위에서 본 대로 헤드를 바꿔도 점수가 같았다.
- **학습 데이터가 너무 적었다.** 학습 사진이 31배가 되자(③) YOLO26m이 YOLO11m과 같은 수준이 됐다. YOLO26의 기본 헤드는 정답 하나에 예측 하나만 짝지어 학습해 정답당 학습 신호가 적다. 사진 193장에서는 이 차이가 크게 드러났을 수 있다(문헌에 기댄 해석이며 직접 검증하지는 않았다).
- **비교 조건도 달랐다.** YOLO26m은 단일 holdout, YOLO11m은 5 fold 분할로 학습했고 GPU도 달랐다. YOLO26m만 ②를 건너뛰어 에폭 효과와 데이터 효과를 나눠 볼 수 없다.

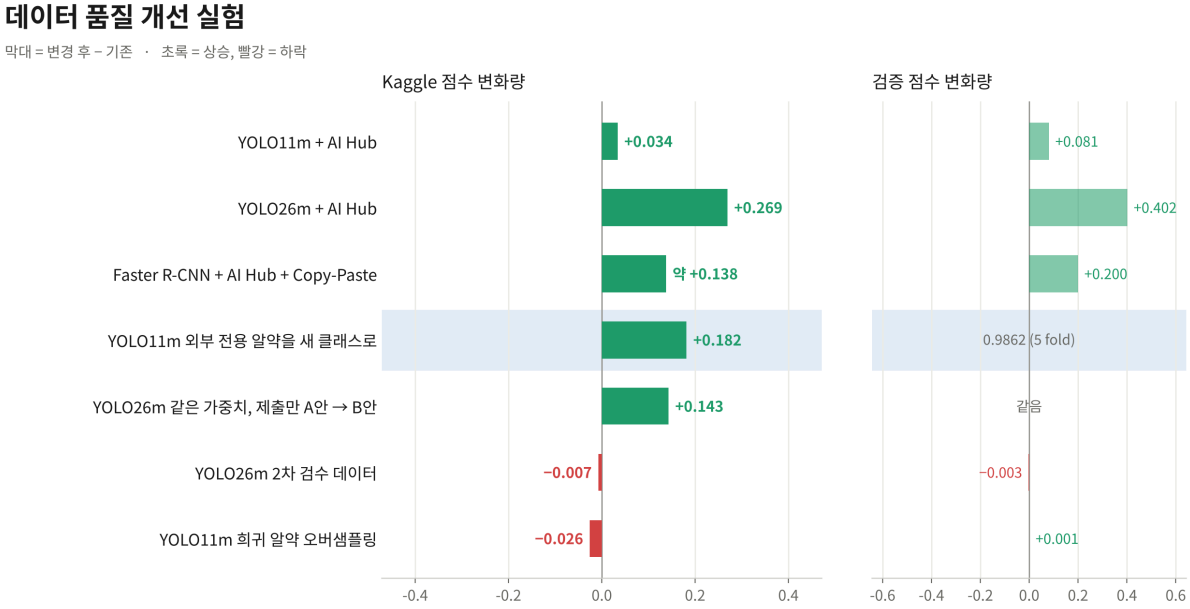
가설별 판정과 남은 검증 실험은 YOLO26m 제외 결정 재검토 문서에 정리했다.

실험별 한계

실험	기대했던 것	한계: 왜 기대만큼 안 됐나
YOLO11m 30 에폭	COCO 사전학습 가중치라 30 에폭이면 충분히 수렴한다	학습이 덜 끝나 Kaggle 0.299에 그쳤다. 검증은 0.84로 높아 검증만 보고는 덜 된 줄 몰랐다
YOLO26m (NMS · NMS-free)	COCO에서 YOLO11m보다 높은(53.1) 최신 구조라 더 잘 나온다	Kaggle 0.18로 가장 낮았다. 기본 헤드로 평가를 바꿔도 0.184였다. 단일 holdout·30 에폭이고 NMS 버전과 같은 조건 비교를 못 해 원인을 끝까지 밝히지 못했다. AI Hub를 넣자 0.454가 되어, 232장으로는 학습이 부족했다고 본다
Faster R-CNN 30 · 100 에폭	two-stage라 박스가 정밀해 IoU 0.75 이상에서 유리하다	100 에폭에도 약 0.32였다. 파라미터 2배·연산 4배라 적은 데이터에서 학습이 더뎠고, 증강과 하이퍼파라미터를 YOLO 기준으로 맞춰 two-stage에 맞는 설정을 찾지 못했다
YOLO11m 100 에폭	더 오래 학습하면 오른다 (기대대로 +0.115)	실험 하나가 GPU로 1-2시간이 되어 이후 시도할 수 있는 실험 수가 줄었다. patience 10 조기 종료라 fold마다 멈춘 에폭이 달랐다

> → **그래서** 이후 모든 실험을 100 에폭·patience 10으로 맞췄고, 후보는 같은 데이터 전략을 적용한 뒤에 비교하기로 했다.

4.3 데이터 품질 개선 실험



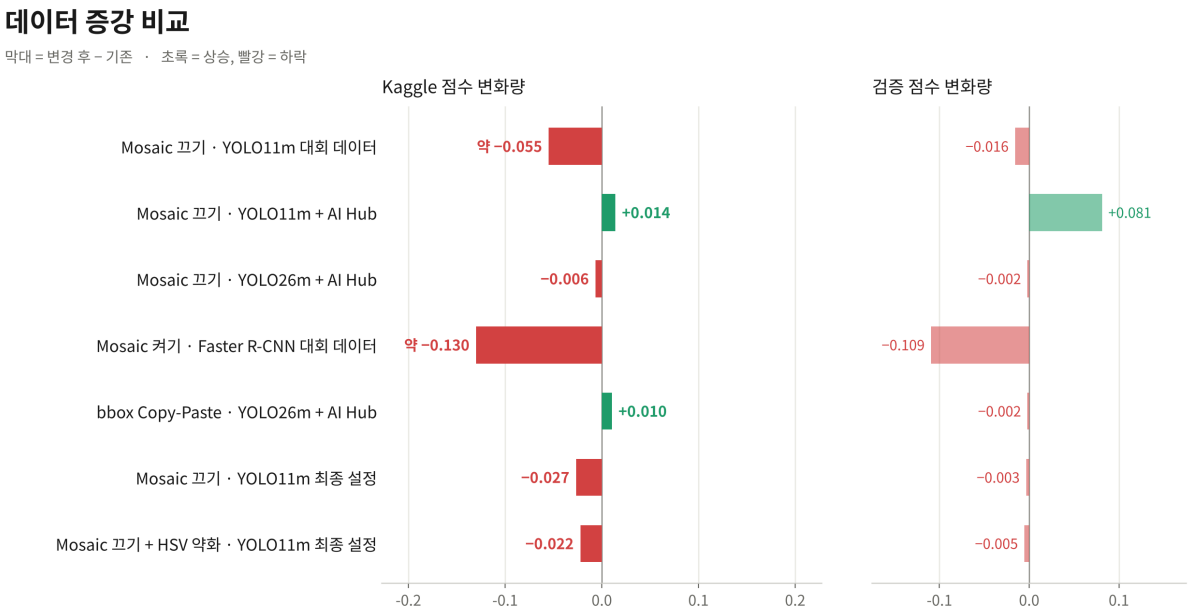
- **외부 데이터가 세 모델 모두를 끌어올렸다.**
- **가장 큰 한 걸음은 외부 전용 알약이었다(+0.182).** 처음에는 대회에 없는 약을 배경으로 뒀다. 그런데 같은 가중치로 확인하니 외부 클래스를 예측한 사진이 대회 사진은 46장 중 1장, test는 842장 중 339장(40%)이었다. test에 대회 밖 알약이 있었던 것이다.
- **라벨 재검수는 병목이 아니었다.** 223개를 고치고 106장을 뺐지만 -0.007이었다.
- **오버샘플링**: fold train에서 bbox가 100개 미만인 약이 든 사진을 2-6배 복제했다(val은 그대로). 검증은 기준과 같은 수준이었다.

실험별 한계

실험	기대했던 것	한계: 왜 기대만큼 안 됐나
AI Hub 보충 (YOLO11m)	희귀 약을 클래스당 157개까지 채우면 크게 오른다	+0.034에 그쳤다. 대회 밖 알약을 배경으로 둬서 test 사진 40%에 있는 그 알약을 맞히지 못했다. 공통 54종 중 2종은 150개를 채우지 못했고, 런타임이 끊겨 검증 지표를 남기지 못했다
외부 전용 알약 새 클래스 · B안 제출	test의 대회 밖 알약까지 맞힌다 (기대대로 +0.14-0.18)	검증 사진에는 외부 알약이 거의 없어(46장 중 1장) 효과를 Kaggle로만 확인했다. Kaggle이 약 코드로 채점한다는 것도 결과에서 거꾸로 추정한 것이다. 96종 밖의 약은 여전히 알아보지 못한다
2차 검수 데이터 (YOLO26m)	라벨 오류 223개를 고치고 문제 사진 106장을 빼면 오른다	-0.007이었다. 라벨 오류는 병목이 아니었고, 뺀 사진만큼 학습 데이터가 줄었다. 한 번만 돌려 ±0.01 노이즈와 구분되지 않는다
희귀 알약 오버샘플링 (YOLO11m)	희귀 약 사진을 2-6배 더 보여주면 희귀 약을 더 잘 맞힌다	같은 사진을 복제할 뿐이라 새 모양·각도 정보가 없다. 검증 사진에 희귀 약 박스가 적어 효과를 잴 수 없었고(0.9658 → 0.9664), Kaggle 점수는 아직 없다

→ **그래서** 외부 전용 알약을 새 클래스로 학습하고 약 코드로 제출하는 방식(B안)을 모든 모델의 기본으로 삼았다.

4.4 데이터 증강 기법 비교



- **Mosaic**: 대회 데이터만 쓸 때는 도움이 됐지만(끄면 -0.055), 외부 데이터를 넣은 뒤(외부 알약 = 배경)에는 끄면 YOLO11m +0.014, YOLO26m -0.006으로 방향이 엇갈렸다. 최종 설정(외부 알약 = 새 클래스)에서는 끄면 -0.027이었다. 끄는 쪽이 낫다는 근거는 없었다.
- **Copy-Paste**: Ultralytics 옵션은 세그멘테이션 마스크가 있어야 동작해 bbox 데이터에서는 아무 일도 하지 않았다. 그래서 정답 bbox를 잘라 빈 배경에 붙이는 방식을 직접 구현했다.
- **Rotate·HSV**: 각인과 색이 약을 구분하는 단서라 처음부터 약하게 두었고, 더 낮춰도 점수는 오르지 않았다.

실험별 한계

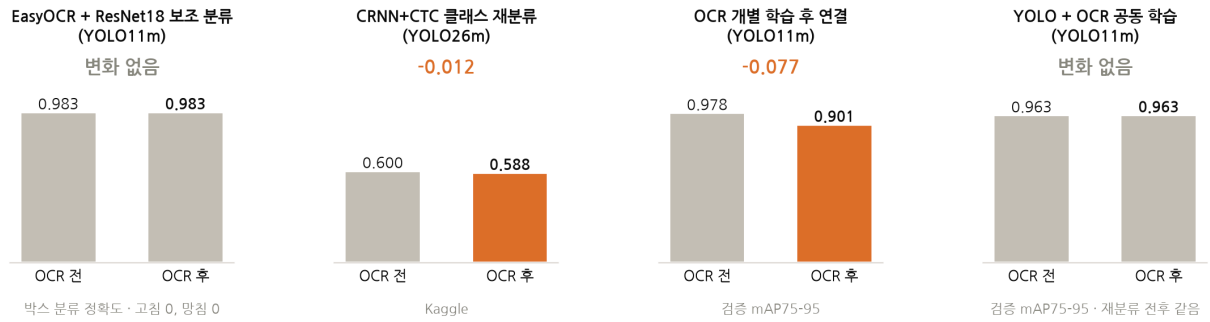
실험	기대했던 것	한계: 왜 기대만큼 안 됐나
Mosaic 끄기 (YOLO11m · YOLO26m)	잘라 붙인 조합 대신 test와 같은 원래 장면으로 학습하면 오른다	모델마다 방향이 반대였다(YOLO11m +0.014, YOLO26m -0.006). GPU가 달랐고(A100 · L4) seed 하나라 노이즈와 구분되지 않는다. 끄면 다른 약 혼동이 14 → 24개, 오탐이 6 → 19개로 늘었다
Mosaic 켜기 (Faster R-CNN)	YOLO에서 통한 Mosaic이 two-stage에도 다양성을 준다	약 0.32 → 0.19로 크게 떨어졌다. 검증은 오히려 올라 검증과 Kaggle이 반대로 움직였다. YOLO 방식을 그대로 옮겨 two-stage에 맞게 조정하지 못했다
bbox Copy-Paste (YOLO26m · Faster R-CNN)	희귀 약 crop을 빈 배경에 붙여 개수를 늘리면 불균형이 준다	+0.010으로 노이즈 수준이었다. 사각형 crop을 그대로 붙여 경계·그림자가 실제 사진과 다르고, AI Hub로 157개를 채운 뒤라 더할 여지가 적었다. Faster R-CNN은 AI Hub와 함께 넣어 Copy-Paste만의 효과를 따로 재지 못했다
Rotate · HSV 약화 (YOLO11m)	색과 각인을 덜 흔들면 비슷한 약을 더 잘 구분한다	처음부터 약하게(회전 10°, 색조 0.005) 둬서 더 줄일 여지가 없었다. Mosaic 끄기와 함께 바꿔 HSV만의 효과를 분리하지 못했고, fold 0 검증 하나로만 비교했다

→ **그래서** 최종 설정은 기본 증강(Mosaic 1.0, 회전 10°)을 그대로 두었다.

4.5 각인 OCR 결합 실험

각인 OCR 결합

네 가지 결합 방식 모두 점수를 올리지 못했다. 각인을 제대로 읽은 박스가 2.5%뿐이었다.



비슷하게 생긴 약을 각인 글자로 구분하려 했지만, EasyOCR이 각인을 정답과 0.8 이상 맞게 읽은 박스는 2.5%뿐이었다. 각인이 작고 음각이라 읽기 어렵고, 잘못 읽은 글자가 맞는 예측을 덮어써 오히려 점수가 내려갔다.

실험별 한계

실험	기대했던 것	한계: 왜 기대만큼 안 됐나
EasyOCR + ResNet18 보조 분류 (YOLO11m)	각인 글자를 읽어 비슷하게 생긴 약의 분류를 고친다	고친 박스가 0개였다. EasyOCR이 각인을 정답과 0.8 이상 맞게 읽은 박스가 2.5%뿐이었다. YOLO가 이미 98.3%를 맞혀 고칠 대상도 적었다
CRNN+CTC 재분류 (YOLO26m)	대회 약 각인으로 직접 학습한 OCR은 더 잘 읽는다	-0.012였다. 각인 자체가 부족했다: 56종 중 8종은 앞면 각인이 없거나 로고(마크)뿐이고, 21종은 글자가 2자 이하다. 'YH'처럼 다른 약이 같은 각인을 쓰기도 한다. 잘못 읽은 글자가 맞는 예측을 덮어썼다
OCR 개별 학습 후 연결 (YOLO11m)	OCR을 따로 잘 학습해 붙이면 YOLO의 실수를 보완한다	검증이 0.978 → 0.901로 떨어졌다. 틀린 글자가 맞는 박스의 클래스를 바꿔 P·R이 0.99대에서 0.92-0.95로 내려갔다. 읽기 품질이 낮은 채 결합 방식만 바꿔서는 오를 수 없었다
YOLO + OCR 공동 학습 (YOLO11m)	함께 학습하면 두 신호가 서로를 보완한다	재분류 전후 0.96322로 같았다. 각인 신호가 약해 재분류가 결과를 바꾸지 못했다

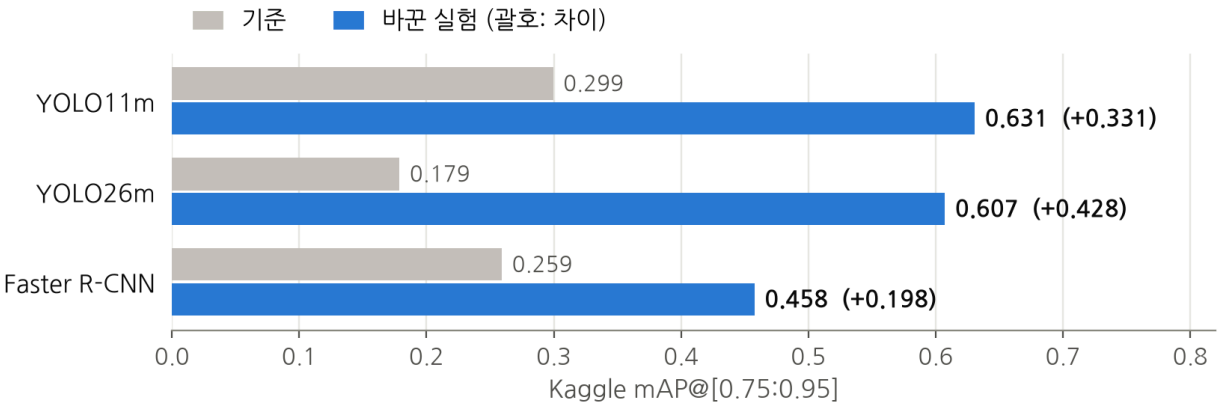
> → **그래서** 최종 모델에서는 OCR을 뺐다. OCR을 쓰기 위해선 각인을 위한 준비가 필요하다고 판단.

4.6 모델별 성능 비교

모델별 베이스라인과 최고 점수

목적: 세 모델을 같은 출발점에서 비교한다. 읽는 법: 회색은 30 에폭 베이스라인, 파랑은 최고 제출이다.

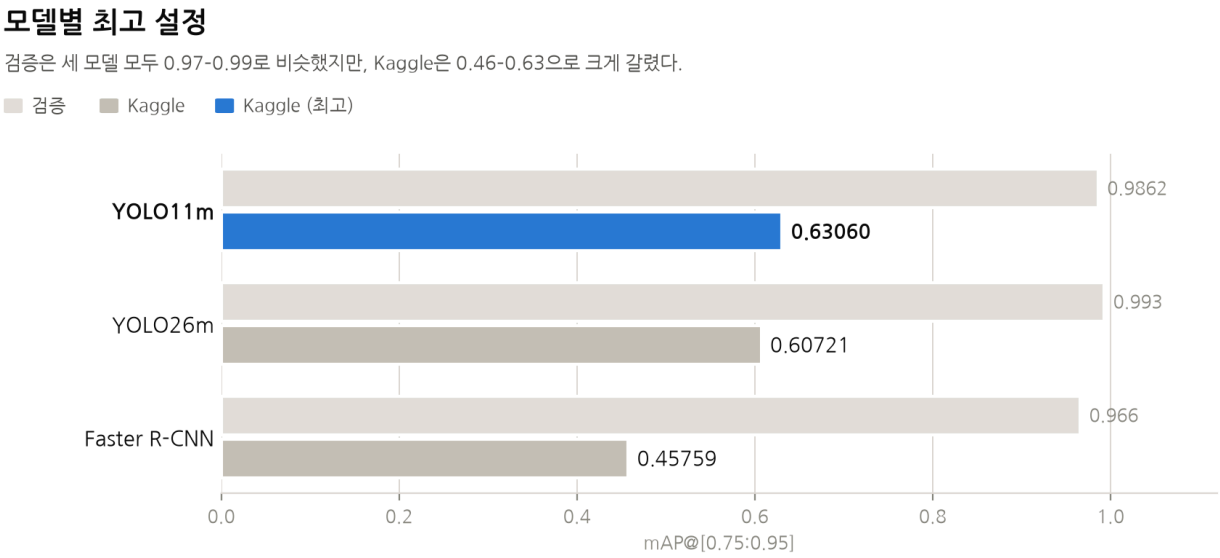
세 모델 모두 2배 이상 올랐고, YOLO11m이 가장 높았다



> *핵심: 같은 데이터 전략이 세 모델 모두에 통했고, 최고 제출은 YOLO11m의 0.631이었다. 다만 YOLO26m과의 차이 (0.023)는 분할·증강·GPU가 달라 정확도 우열로 보지 않는다(3.5).*

모델별 최고 설정

모델	최고 설정	검증	Kaggle
YOLO11m	AI Hub + 외부 전용 클래스	0.9862	**0.63060**
YOLO26m	AI Hub + Copy-Paste + 외부 전용 클래스	0.993	0.60721
Faster R-CNN	AI Hub + Copy-Paste	0.966	0.45759



모델별 한계

실험	기대했던 것	한계: 왜 기대만큼 안 됐나
YOLO11m	속도·정확도 균형이 좋고 앱 변환이 쉽다 (최고 0.631)	검증 0.986과 Kaggle 0.631의 차이가 커서 검증으로는 실제 성능을 알 수 없다. max_det 4라 사진에 알약이 5개 이상이면 놓친다. int8 변환 후 정확도는 대회 사진으로만 확인했다
YOLO26m	COCO 최고 구조라 가장 높을 것이다	YOLO11m보다 0.023 낮았다. 검증이 일찍 상한(0.995)에 닿아 best.pt와 조기 종료를 검증으로 고를 수 없었고(best 15-24 에폭), 데이터가 적을 때 가장 약했다
Faster R-CNN	two-stage의 박스 정밀도로 높은 IoU에서 앞선다	최고 0.458로 YOLO보다 0.15 이상 낮았다. 외부 전용 알약 새 클래스(B안) 노트북은 만들었지만 결과를 얻지 못했다. 추론이 가장 느리고(21 ms) 앱 변환 경로가 없다

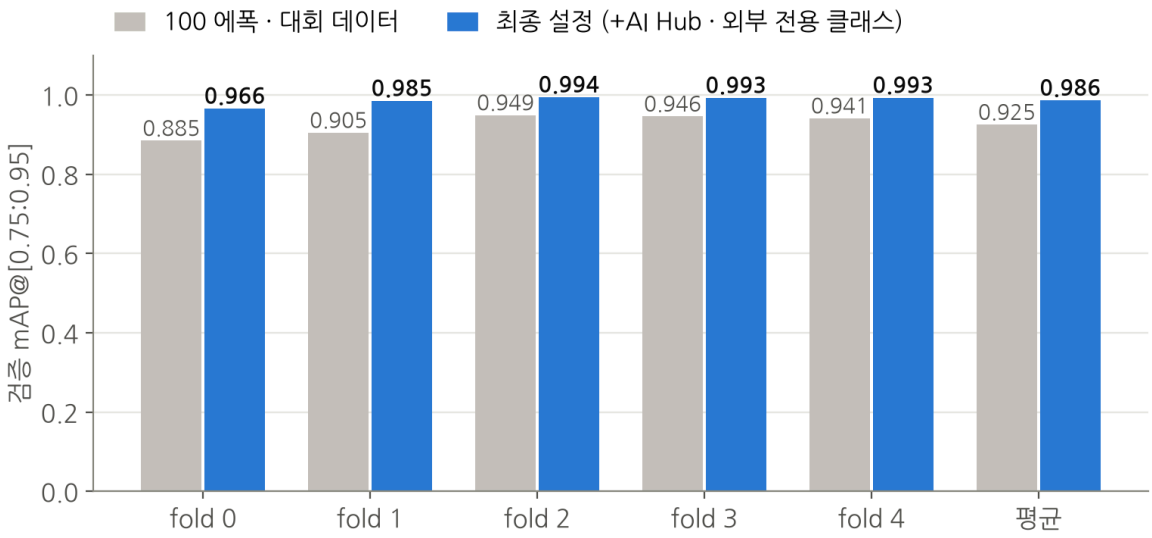
4.7 최종 모델 성능

최종 모델

항목	값
모델	YOLO11m, 96종(대회 56 + 외부 전용 40), 전체 파인튜닝

항목	값
데이터	대회 train + AI Hub 선별 사진 5,842장. val은 대회 사진만
검증 (5 fold)	**mAP50 0.9912, mAP50-95 0.9887, mAP75-95 0.9862 ± 0.0120**
Kaggle	**0.63060 (베이스라인 0.29946의 2.1배)**. 5 fold 각각의 best.pt로 낸 제출 중 가장 높았던 fold 1의 점수다. 나머지 fold의 점수는 저장소에 남아 있지 않다
배포	int8 20MB, 웹(ONNX)에서 기기 안 추론. Android·iOS는 검증하지 않았다

최종 설정은 다섯 fold 모두 높고 편차가 작다



> *핵심: 다섯 fold 모두 올랐고, 표준편차가 0.029에서 0.012로 줄었다.*

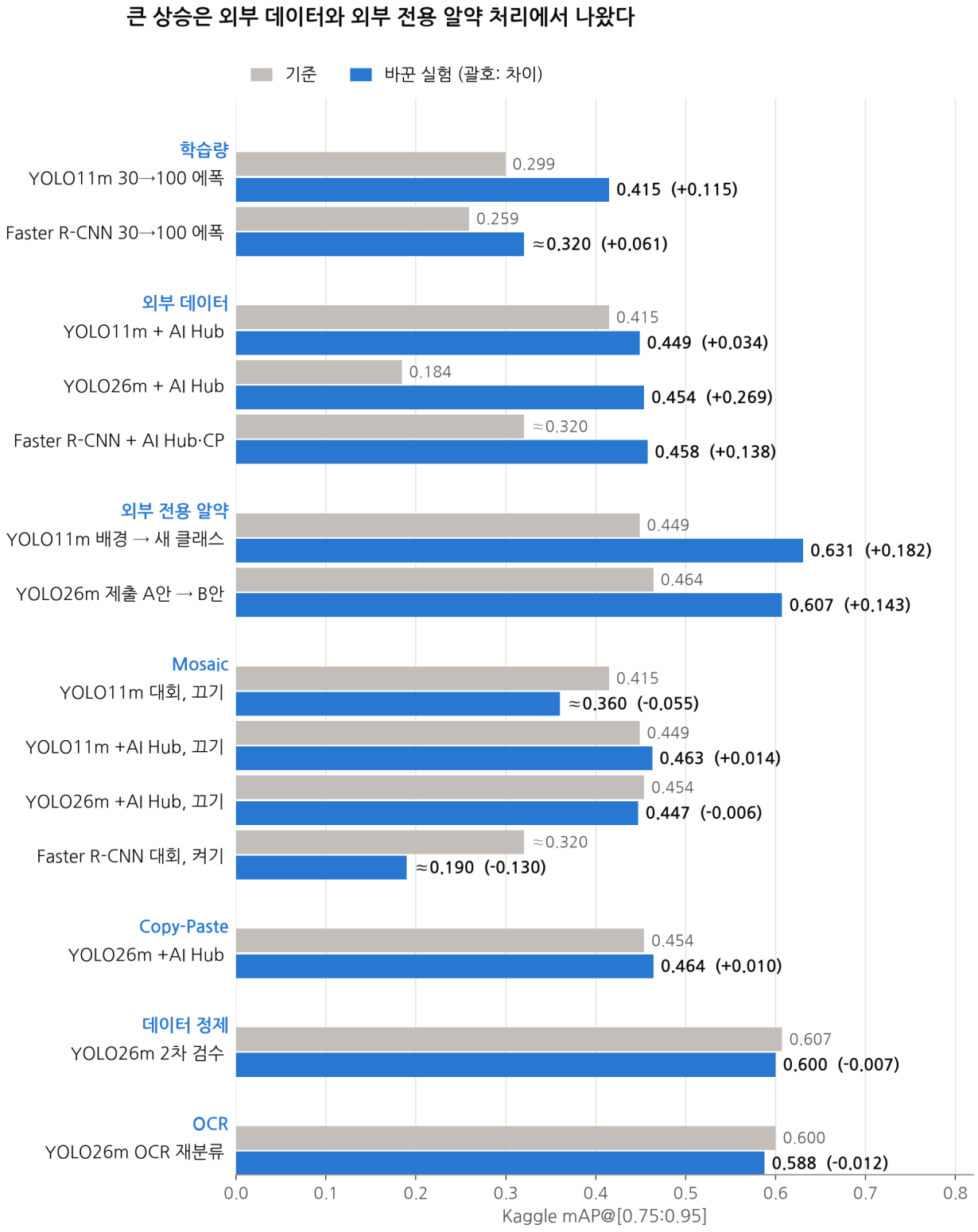
이 값은 검증 점수다. 5.2에서 보듯 검증은 0.99 근처에서 포화돼 있어, fold마다 학습이 고르게 수렴했다는 근거일 뿐 실제 사진에서의 성능을 보장하지는 않는다. 서비스 관점의 안정성 근거는 5.5에 모았다.

5장. 결과 분석 및 시각화

한 일	실험 결과를 목적별로 모아 보고, 검증 점수와 Kaggle 점수가 왜 어긋나는지 따져 봤다.
이유	검증이 0.99에서 포화되어 어떤 실험이 진짜 나은지 검증만으로는 알 수 없었다.
결과	데이터가 성능을 결정했고, 모델 선택은 Kaggle 점수로 해야 한다는 결론을 얻었다.

5.1 무엇이 점수를 올렸나

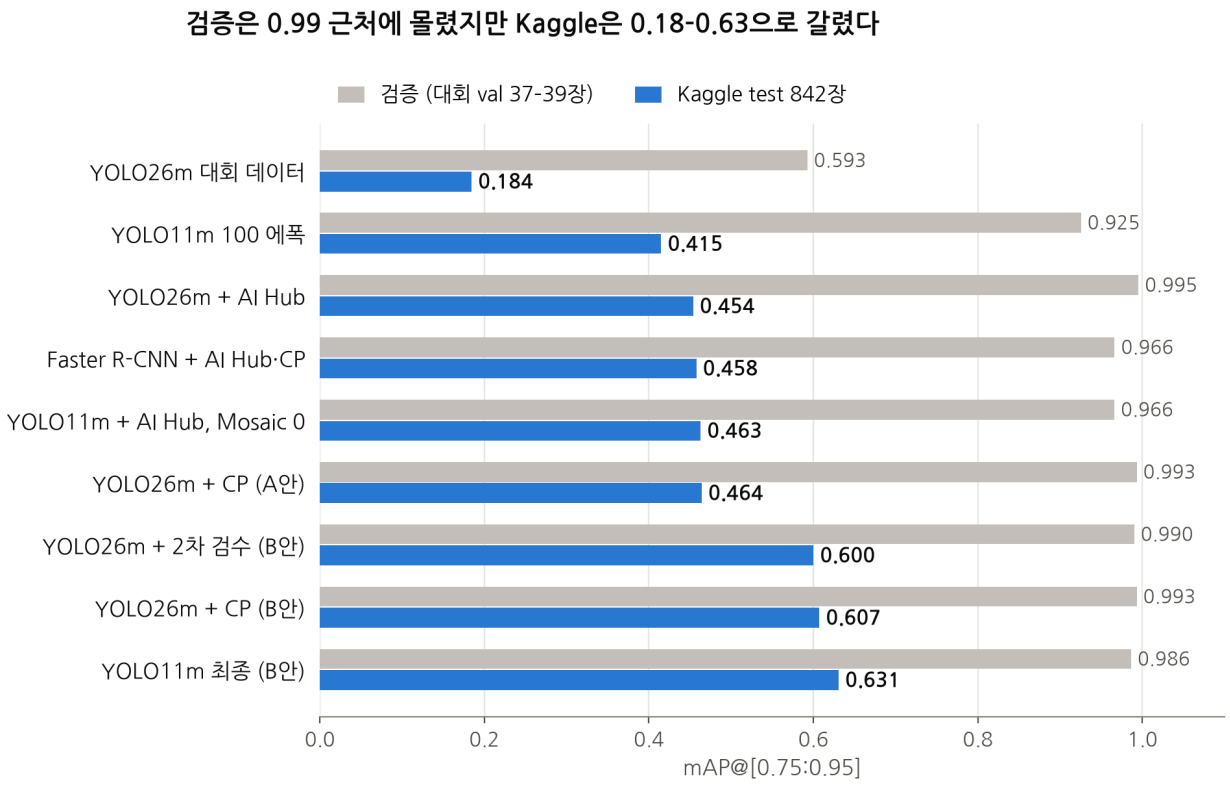
실험 목적별 Kaggle 점수 변화



핵심: +0.1 이상은 모두 데이터 쪽(학습량, 외부 데이터, 외부 전용 알약)에서 나왔다. 외부 데이터를 쓴 뒤의 증강·정제·OCR은 -0.027에서 +0.014 사이였다.

5.2 검증 점수를 믿을 수 없었던 이유

같은 실험의 검증 점수와 Kaggle 점수



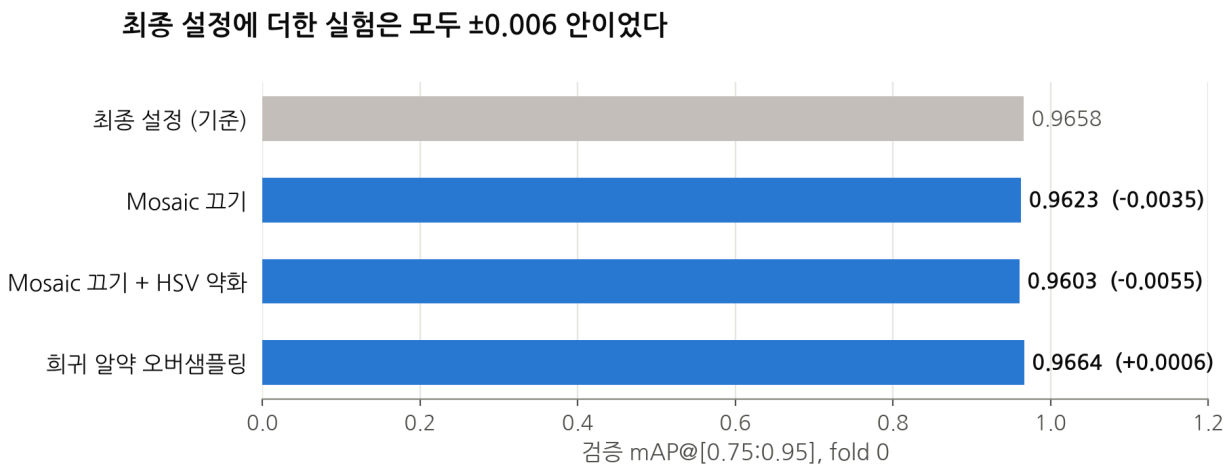
-왜 이렇게 차이가 났을까?

- **검증이 작다**: fold 0 검증은 37장(fold에 따라 37-40장), 박스 117개다.
- **누수가 남았다**: 조합 단위로 나눠도 세팅이 같은 사진이 48-75% 남는다.
- **검증에 없는 약**: test 사진의 약 40%에 대회 밖 알약이 있다.
- **포화**: YOLO26m은 24 에폭부터 모든 클래스가 상한(0.995)이었다.

> → **그래서** 실험 비교와 모델 선택은 Kaggle 점수로 했다.

5.3 최종 설정 위 추가 실험

최종 설정(yolo11m 베스트 모델)에 한 가지씩 더한 실험

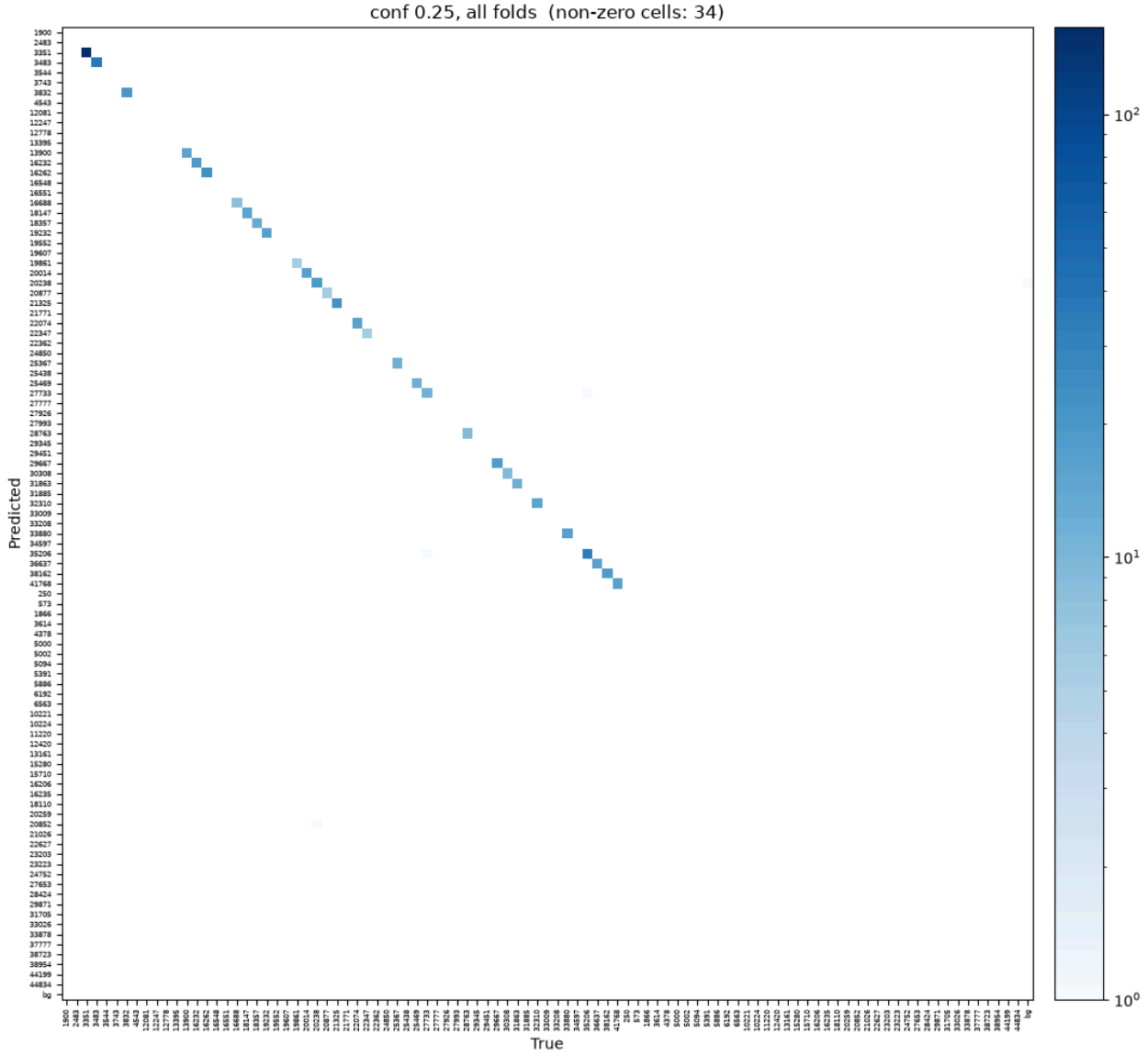


> *핵심: 검증 점수는 모두 ±0.006 안이었다. Kaggle 점수는 Mosaic 끄기 -0.027, Mosaic 끄기 + HSV 약화 -0.022(4.4), 희귀 알약 오버샘플링 -0.026(4.3)으로 모두 0.63060보다 낮았다. 최종 설정에서 증강을 바꿔 더 오른 실험은 없었다.*

5.4 혼동행렬과 과적합

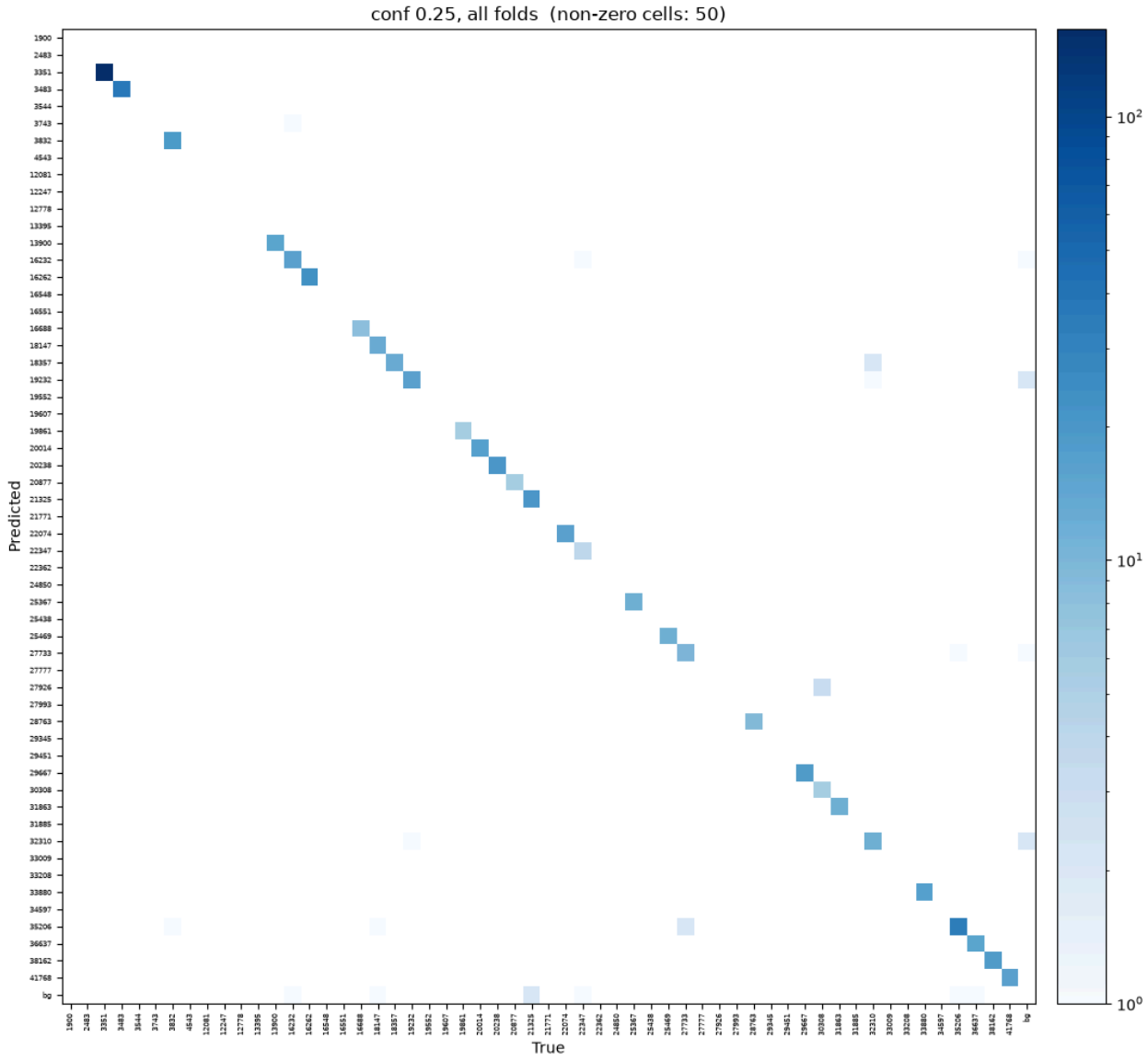
최종 모델 혼동행렬 (AI Hub + 외부 전용 클래스, 5 fold 검증 합계, conf 0.25)

목적: 어떤 약을 무엇으로 헷갈리는지 본다. 읽는 법: 행은 예측, 열은 정답, 대각선이 맞힌 것이고 마지막 bg는 배경이다.



> *핵심: 최종 모델은 615개 중 612개(99.5%)를 맞혔다. 다른 약 혼동 3개, 놓침 0개, 오탐 1개다.*

100 에폭 베이스라인 혼동행렬 (대회 사진만, 5 fold 검증 합계, conf 0.25)



> *핵심: 외부 데이터를 넣기 전 베이스라인은 615개 중 594개(96.6%)를 맞혔다. 다른 약 혼동 14개, 놓침 7개, 오탐 6개다.*

- 베이스라인에서 Mosaic을 끄면 다른 약 혼동이 14 → 24개, 오탐이 6 → 19개로 늘었다.
- conf는 0.25로 고정했다. 0.001에서는 낮은 점수 박스가 섞여 0이 아닌 칸이 31개에서 55개로 늘어 읽기 어렵다.
- **과적합 점검**: 최종 설정 fold 0의 train-val 차이는 0.0255였다. val cls_loss가 최저 대비 24.9% 다시 올랐지만 mAP는 떨어지지 않았다.

5.5 서비스에 넣어도 되는가: 안정성 근거와 남은 위험

알약몬에서 모델이 틀리면 복용 기록과 보상이 함께 틀린다. 그래서 점수 하나가 아니라 "틀릴 때 얼마나, 어떻게 틀리는가"와 "틀려도 사용자가 바로잡을 수 있는가"를 함께 봤다.

근거	내용
틀리는 양과 방식	최종 모델은 5 fold 검증 합계 615개 중 612개(99.5%)를 맞혔다. 놓침 0개, 오탐 1개, 다른 약으로 혼동 3개다. 대회 사진만 쓴 베이스라인은 594개(96.6%)였다(5.4). 검증 사진으로 잰 값이라 실제 사진의 성능은 아니다(5.2)
과적합	최종 설정 fold 0의 train-val 차이는 0.0255였다(5.4)
경량화 뒤에도 같은 결과	가중치를 int8로 저장해 모델 파일을 80MB에서 20MB로 줄였고, 탐지 결과는 원래 모델과 같았다
앱의 안전장치	신뢰도 0.5 미만은 약으로 보지 않는다. 한 알약에 서로 다른 약 박스가 겹치면 하나만 남기고 나머지는 바꿀 후보로 둔다. 모델이 놓친 물체는 색으로 따로 찾아 "알 수 없는 물체"로 보여 준다(대회 사진 알약의 99%). 모든 결과는 사용자가 확인·수정한 뒤에 저장된다

남은 위험

- **휴대폰으로 찍은 실제 사진의 성능은 아직 재지 않았다.** 검증과 test 모두 대회 촬영 환경의 사진이다. 다음 검증 1순위다(6.4).
- **학습하지 않은 약은 인식하지 못한다.** 비슷한 다른 약으로 잡히기 쉬워, 앱은 사용자가 고른 약의 색·모양을 기억해 다음 사진에서 바꾸자고 제안한다.
- **검증 점수로는 우열을 가릴 수 없다.** 검증이 포화돼 있어(5.2) 모델 비교는 Kaggle 점수 하나에 기대고 있다.

5.6 종합

- **데이터가 모델보다 중요했다.** 희귀 약이 3개뿐인 데이터에서는 증강보다 실제 사진을 더하는 것이 효과가 컸다.
- **test를 알아야 했다.** test의 40%에 대회 밖 알약이 있었고, 그 약을 학습·제출하자 +0.14-0.18이 올랐다. 외부 데이터를 더 끌어와야 했다.
- **OCR은 읽기 품질이 먼저다.** 인식률 2.5%에서는 결합 방식과 관계없이 오르지 않는다.

6장. 결론 및 향후 과제

6.1 결론

클래스 불균형을 외부 데이터로 돌파한 것이 이 프로젝트의 핵심이었다. 우리는 희귀 약을 157개까지 채우고 대회 밖 알약까지 학습해 YOLO11m의 Kaggle 점수를 0.29946에서 0.63060으로 2.1배 올렸고, 이 모델을 int8 20MB로 줄여 웹 앱에서 기기 안 추론으로 배포했다. 라벨 보정, 분할, 외부 데이터 선별·검수를 모두 git에 기록해 같은 결과를 다시 만들 수 있다.

정확한 검출은 알약몬 보상 시스템의 전제다. 어떤 약을 몇 알 먹었는지 맞게 알아야 하루 기준량 안에서만 보상을 줄 수 있다.

6.2 우리가 배운 것

회고: 한 일 → 배운 점 → 다음 행동

이렇게 했더니	이걸 알게 됐고	다음엔 이렇게
증강보다 외부 데이터로 불균형을 먼저 잡음	희귀 약은 증강보다 실제 사진이 효과가 크다	데이터 양·분포부터 확인하고 증강은 나중에
검증 점수로 실험을 비교함	누수와 작은 검증 때문에 0.99에서 포화된다	처음부터 세팅 단위 분할·5 fold로 검증 설계
대회에 없는 약을 배경으로 둠	test에는 대회 밖 알약이 40% 사진에 있었다	베이스라인을 최우선적으로 만들고 test에 대한 모델 성능을 일찍 확인
Google Drive로 데이터를 받음	다운로드 제한에 걸려 실험이 반나절 넘게 밀렸다	데이터는 시작 직후 로컬·Kaggle Dataset으로
커널이 죽으면 처음부터 다시 학습	GPU 시간과 일정을 낭비했다	last.pt 이어 학습·자동 종료 코드를 기본으로

6.3 한계

실험마다의 한계는 4장 각 절의 '실험별 한계'·'모델별 한계' 표에 적었다. 여기에는 프로젝트 전체에 걸친 한계만 모았다.

- 대부분 fold 0, seed 하나로 한 번씩 돌린 결과라 ±0.01 차이는 노이즈와 구분되지 않는다.
- 실험마다 GPU(A100, L4, T4, RTX 4070 Ti SUPER)와 배치가 달라 속도와 일부 점수를 직접 비교하기 어렵다.
- 휴대폰으로 찍은 실제 사진의 성능은 아직 재지 않았다.
- **후보를 베이스라인 하나로 뺐다.** YOLO26m을 30 에폭 점수 하나로 제외했다가 다시 넣었다. 논문을 근거로 고른 후보는 같은 데이터 전략을 적용한 뒤에 판단했어야 했다(4.2).

- **실험 순서에 가설을 반영하지 못했다.** "데이터가 병목"이라는 가설을 3.2에서 세웠지만, 외부 데이터 이후에도 증강 실험을 여럿 이어 갔고 대부분 노이즈 범위의 결과로 끝났다(5.1).
- Ultralytics 모델은 AGPL-3.0이라 공개 배포 전 소스 공개나 Enterprise License를 정해야 한다.

6.4 향후 과제

- 알약몬 보상 시스템을 구현한다: 처방전 OCR로 복용 일정 등록, 캐릭터 성장·질병 몬스터, 하루 기준량을 넘은 기록은 보상에서 제외 등등.
- 전체 데이터 재학습을 다시 설계한다. 11.05에 검증 없이 전체 데이터(train + val)로 seed 5개를 학습해 제출했지만 0.63060을 넘지 못했다. 20 에폭에서 멈춘 last.pt는 seed마다 test 예측 박스 수가 3,541-11,490개로 크게 달랐다. 멈출 에폭을 정하는 기준부터 세운다.
- 앱에서 동의를 받아 모은 실제 사진으로 재학습한다.
- **실제 휴대폰 사진으로 성능을 잰다.** 배경·조명이 다른 사진에서 놓침과 오탐을 센다(5.5).
- **정확도 우열을 확정한다.** YOLO11m과 YOLO26m을 같은 분할·같은 GPU에서 seed 3개씩 다시 비교하고, YOLO26m의 대회 사진만 100 에폭 실험을 채운다. 두 모델을 앱 형식(Core ML·TFLite)으로 내보내 같은 기기에서 추론 시간도 잰다.
- **Faster R-CNN을 다시 잰다.** 증강을 YOLO 기준으로 맞춰 손해를 봤을 수 있어, two-stage에 맞춘 증강으로 다시 비교한다.
- 5 fold·여러 seed·세팅 단위 분할로 증강 효과를 다시 재고, WBF 앙상블·TTA를 검토한다.

부록

A. 사용 기술

사용 기술

분야	기술
모델	PyTorch 2.14, Ultralytics 8.4 (YOLO11m, YOLO26m, SAM), torchvision Faster R-CNN
데이터	pandas, NumPy, OpenCV(GrabCut, HSV), pHash
OCR	EasyOCR, ResNet18, CRNN+CTC
배포	Flutter, onnxruntime-web, TFLite, Core ML, FastAPI, SQLite, GitHub Actions, Cloudflare Tunnel
환경 · 협업	Python 3.12, uv, Colab·Kaggle GPU, W&B, Git·GitHub PR, ruff, Notion, Discord

B. 근거 PR

- EDA 통합, 데이터 파이프라인: #38, #56
- AI Hub EDA, 전처리, 2차 검수: #93, #104, #116
- 세 모델 베이스라인, 100 에폭: #98-#103
- 외부 데이터, Mosaic, 외부 전용 알약: #105-#109
- YOLO26m NMS-free, Copy-Paste, OCR: #110-#114, #120
- YOLO11m OCR, HSV, Rotate, Mosaic: #117, #121-#123
- Faster R-CNN 외부 데이터 + Copy-Paste: #118, #119

C. 참고 자료

- 모델 저장소, 앱 저장소, Kaggle 대회
- 팀 Notion(회의록, 일일 기록), 팀 Discord 실험 채널
- Ultralytics YOLO26 (arXiv 2606.03748), Faster R-CNN (arXiv 1506.01497), ViTDet 벤치마크 (arXiv 2111.11429)
- AI Hub 경구약제 이미지 데이터
- YOLO26m 제외 결정 재검토, Wang 등, YOLOv10: Real-Time End-to-End Object Detection (arXiv 2405.14458)

D. 수정 이력

2026.11.08, 발표 피드백을 반영해 앞뒤가 맞지 않던 부분을 고쳤다. 실험 결과 수치는 바꾸지 않았다.

- **모델 선정 근거**: "앱 변환의 쉬움"을 근거에서 뺐다. 같은 보고서가 YOLO26의 변환이 더 쉽다고 적었기 때문이다. 근거를 "최고 제출 + 앱 연동 완료"로 고치고 3.5를 새로 썼다.
- **YOLO26m 베이스라인 해석**: "같은 출발점에서 비교했다"를 고쳤다. 분할과 실행 환경이 달랐다는 점, 같은 데이터 전략에서는 차이가 없었다는 점을 4.2에 넣었다.
- **노이즈 범위**: 6.3에서 ±0.01을 노이즈라고 적은 기준에 맞춰, ±0.014 안의 증강·OCR 결과를 "효과 없음"이 아니라 "노이즈와 구분되지 않음"으로 고쳤다.
- **검증 점수의 쓰임**: 5.2에서 검증을 믿을 수 없다고 한 것과 맞춰, 4.7의 fold 편차는 수렴의 근거로만 쓰고 안정성 근거는 5.5에 따로 모았다.
- **클래스 수**: 초록의 "56종"에 앱 모델의 96종을 함께 적었다.
- **구성**: 3장의 절 번호를 목차와 맞추고, 3.1과 3.2에 중복되던 논문 소개를 3.2로 합쳤다. 6.3 한계와 6.4 향후 과제를 위 내용에 맞춰 고쳤다.

2026.11.09, 앱 연동 범위를 실제에 맞췄다. 실험 결과 수치는 바꾸지 않았다.

- **앱 연동 범위**: 연동과 검증을 마친 것은 웹(ONNX)뿐이다. 1.3, 3.5, 6.1에 "웹 · Android · iOS"로 적었던 것을 웹으로 고치고, Android·iOS는 검증하지 않았다고 적었다.

2026.11.11, 본문과 그림·노트북 출력이 어긋나던 부분을 고쳤다. Kaggle 점수는 바꾸지 않았다.

- **혼동행렬**: 5.4와 5.5의 "615개 중 594개"는 최종 모델이 아니라 대회 사진만 쓴 100 에폭 베이스라인의 값이었다. 최종 모델의 혼동행렬(612개)을 더하고, 5.5의 근거를 최종 모델 값으로 바꿨다.
- **증강·정제·OCR의 범위**: 초록, 4장, 5.1의 "±0.014 안"과 4.4의 "±0.015 안"을 4.3·4.4 그림의 값에 맞춰 "−0.027에서 +0.014 사이"로 고쳤다. 5.3의 "±0.006 안"은 검증 점수라는 점과 같은 실험의 Kaggle 변화를 함께 적었다.
- **최종 모델 표**: 4.7 표에 남아 있던 "웹·Android·iOS"를 웹으로 고치고, 0.63060이 5 fold 제출 중 fold 1의 점수라는 것을 적었다. 표지와 초록의 "Flutter 앱"도 "Flutter 웹 앱"으로 맞췄다.
- **향후 과제**: 6.4의 "전체 데이터 재학습"은 11.05에 이미 해 본 실험이라 그 결과를 적었다.
- **수치와 표기**: 두 YOLO의 최고 제출 차이를 0.024에서 0.023(0.63060 − 0.60721)으로, 불균형 CV를 2.4 표와 같은 0.100으로, 최대/최소 배수를 v1 라벨 기준 52배로 고쳤다. 2.5의 라벨 보정 개수는 보정 기록(corrections.csv)의 분류대로 다시 적었다. 3.3의 없는 참조, 3.4의 마크다운 기호와 소제목, 4.2 표의 근거 PR, 5.2의 검증 사진 수를 바로잡았다. 4.6 "모델별 한계"와 4.7 "최종 모델" 표는 그림에서 글자 표로 바꿨다.